

Proiect SGBD

Nume: Pădureanu Maria-Andreea

Grupa: 1067

Seria: E

Cuprins

A. Descrierea problemei.....	3
a. Obiectivul proiectului.....	3
b. Descrierea tabelor și a atributelor.....	3
c. Precizarea restricțiilor și a tipurilor de legături.....	6
B. Schema bazei de date.....	8
C. Interogări.....	9
a. Structuri de control.....	9
b. Gestionarea cursorilor (impliciți și expliți).....	15
c. Tratarea excepțiilor (implicite și explicite).....	22
d. Funcții, proceduri, pachete.....	29
e. Triggers.....	56

GESTIUNEA UNEI FARMACII

A. Descrierea problemei

a. Obiectivul proiectului

Am ales să fac baza de date a unei farmacii deoarece implementarea acesteia poate ajuta o farmacie să organizeze și să gestioneze eficient informațiile legate de stocurile de medicamente, furnizori, vânzări, clienți și rețete. Aceasta permite farmacistului să urmărească disponibilitatea produselor, să actualizeze stocurile în timp real și să îmbunătățească serviciile oferite pacienților, asigurându-se că toate datele sunt accesibile rapid și corecte.

b. Descrierea tabelelor și a atributelor

În baza de date pe care am creat-o am ales să includ următoarele tabele, împreună cu atributele aferente fiecăreia:

- **MEDICAMENTE** - în această tabelă se vor stoca date despre medicamentele existente în farmacie precum: id medicament, nume, categorie, preț, stoc, producător și data de expirare. Se folosește pentru a gestiona inventarul medicamentelor, actualizarea stocurilor și listarea produselor disponibile.

- **CATEGORII_MEDICAMENTE** - tabela va stoca id-ul fiecărei categorii și numele categoriilor de medicamente din farmacie. Facilitează clasificarea medicamentelor pentru căutare rapidă și organizare eficientă.

- FURNIZORI_MED - tabela stochează date despre furnizorii medicamentelor precum id furnizor, nume, adresă, telefon, mail. Ajută la gestionarea relațiilor cu furnizorii, inclusiv contactarea acestora pentru comenzi sau rezolvarea problemelor de aprovizionare.

- FUNCTII_FARMACIE - această tabelă va stoca id-urile și numele funcțiilor pe care le pot avea angajații pentru identificarea rolurilor fiecăruia.

- ANGAJATI_FARMACIE - se vor stoca date despre fiecare angajat al farmaciei precum: id angajat, nume, prenume, telefon, mail, adresă, data de angajare, funcția pe care o deține și salariul. Tabela este utilizată pentru gestionarea resurselor umane, atribuirea responsabilităților și calcularea salariilor.

- ACHIZITII_MEDICAMENTE - în această tabelă se vor stoca date despre fiecare achiziție de medicamente făcută de farmacie de la furnizori. Datele din tabelă se vor referi la furnizorul de la care s-a făcut achiziția, angajatul care a făcut achiziția și data la care s-a făcut aceasta, achiziția fiind identificată printr-un id. Se folosește pentru a urmări aprovizionările și pentru a controla stocurile din inventar.

- RAND_ACHIZITII_MED - în această tabelă se vor păstra date specifice pentru fiecare achiziție în parte precum: medicamentele incluse în fiecare achiziție, prețul unitar al acestora și cantitatea achiziționată. Permite înregistrarea precisă a fiecărui medicament primit într-o achiziție, asigurând transparență și gestionarea corectă a stocului.

- CLIENTI_FARMACIE - tabela va înregistra date despre clienții farmaciei precum: nume, prenume, telefon, mail, data de naștere, sex și adresă. Facilitează gestionarea relațiilor cu clienții și identificarea lor pentru vânzări, rețete și oferte personalizate.

- VANZARI_MEDICAMENTE - înregistrează tranzacțiile de vânzare a medicamentelor către clienți prin stocarea datelor precum: data la care a fost făcută vânzarea, clientul care a cumpărat și angajatul care a făcut vânzarea. Se folosește pentru a urmări vânzările zilnice, a calcula veniturile și a analiza performanța farmaciei.

- RAND_VANZARI_MED - detaliază medicamentele vândute în cadrul unei tranzacții prin stocarea datelor precum: cantitate și preț unitar pentru fiecare medicament inclus în tranzacție. Permite identificarea produselor vândute într-o tranzacție și ajută la actualizarea stocurilor.

- RETETE - înregistrează rețetele emise clienților de medici prin stocarea următoarelor date: id rețetă, numele pacientului, id client (pentru cei care sunt deja clienți ai farmaciei, dar nu au avut rețete), numele medicului care a emis rețeta, data de emitere și data de expirare, dacă rețeta este compensată.

- MEDICAMENTE_RETETA - detaliază medicamentele asociate fiecărei rețete prin stocarea numelor medicamentelor prescrise și cantitatea acestora.

c. Precizarea restricțiilor și a tipurilor de legături

Restricțiile aplicate în tabele sunt:

- Cheie primară - le-am aplicat pe coloanele ce identifică unic fiecare înregistrare din tabel:

id_med → MEDICAMENTE

id_categorie → CATEGORII_MEDICAMENTE

id_furnizor → FURNIZORI_MED

id_achizitie → ACHIZITII_MEDICAMENTE

id_vanzare → VANZARI_MEDICAMENTE

id_client → CLIENTI_FARMACIE
id_reteta → RETETE
id_angajat → ANGAJATI_FARMACIE
id_functie → FUNCTII_FARMACIE

- Cheie externă - care asigură integritatea referențială între tabele:

MEDICAMENTE:

id_categorie → CATEGORII_MEDICAMENTE (id_categorie)

ACHIZITII_MEDICAMENTE:

id_furnizor → FURNIZORI_MED (id_furnizor)

id_angajat → ANGAJATI_FARMACIE (id_angajat)

RAND_ACHIZITII_MED:

id_achizitie → ACHIZITII_MEDICAMENTE (id_achizitie)

id_med → MEDICAMENTE (id_med)

VANZARI_MEDICAMENTE:

id_angajat → ANGAJATI_FARMACIE (id_angajat)

id_client → CLIENTI_FARMACIE (id_client)

RAND_VANZARI_MED:

id_vanzare → VANZARI_MEDICAMENTE (id_vanzare)

id_med → MEDICAMENTE (id_med)

RETETE:

id_client → CLIENTI_FARMACIE (id_client)

MEDICAMENTE_RETETA:

id_reteta → RETETE (id_reteta)

id_med → MEDICAMENTE (id_med)

ANGAJATI_FARMACIE:

id_functie → FUNCTII_FARMACIE (id_functie)

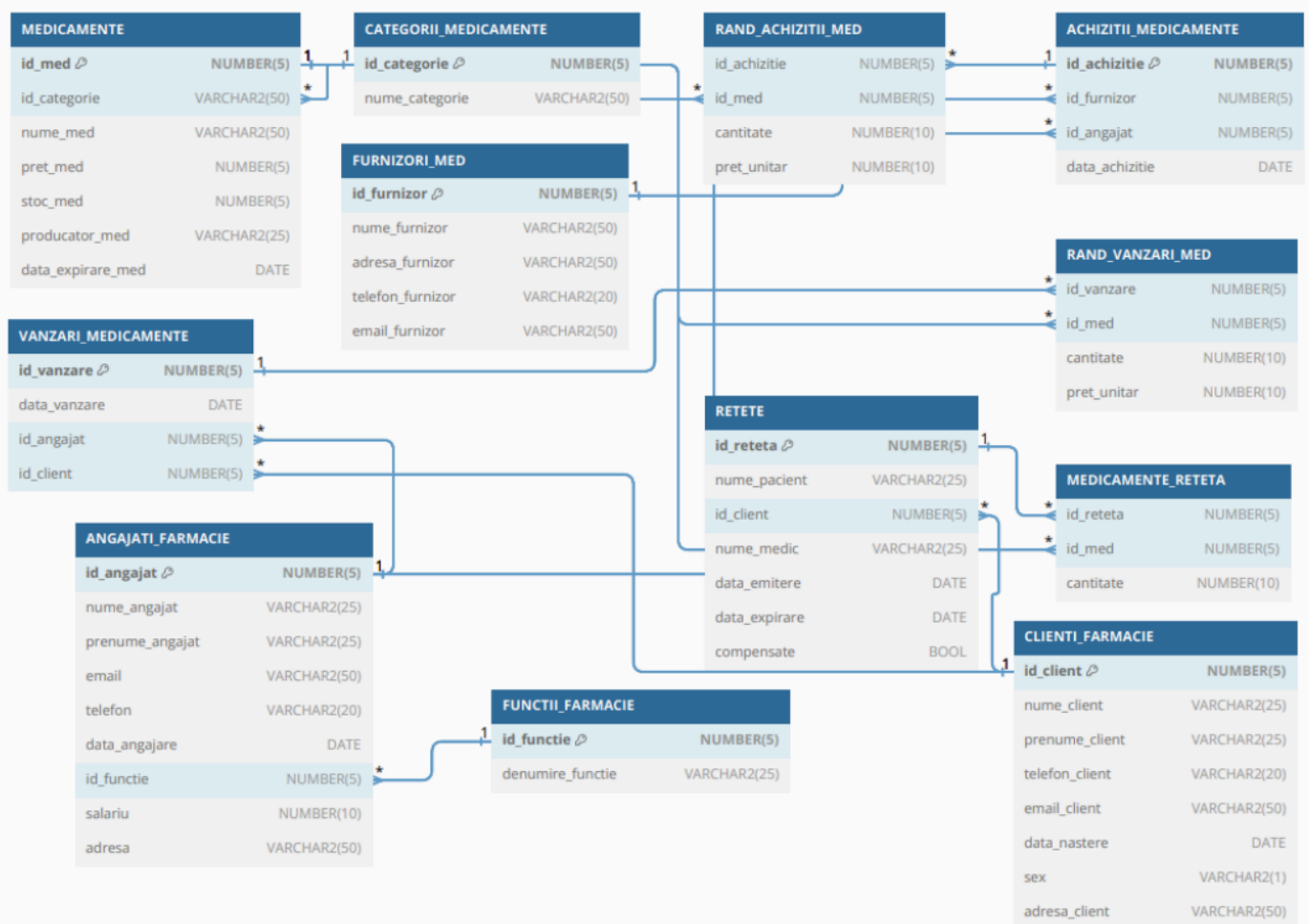
- Alte restricții

NOT NULL – cheile primare și externe trebuie să aibă valori nenule, iar alte coloane care au setată restricția sunt: nume_med, data_expirare_med, nume_furnizor, nume_angajat, prenume_angajat, data_angajare, nume_pacient, nume_medic, data_emitere și data_expirare.

UNIQUE – se aplică pentru cheile primare și externe care trebuie să fie unice în cadrul fiecărei tabele. Le-am mai aplicat și coloanelor de mail din tabelele unde se regăsește.

CHECK – am aplicat restricția pentru atributul compensate din tabela rețete pentru a verifica dacă valoarea este 'DA' sau 'NU'.

B. Schema bazei de date



C. Interogări

a. Structuri de control

- Generare raport medicamente aproape de expirare (în următoarele 30 de zile).

```
set serveroutput on
```

```
begin
```

```
    dbms_output.put_line('medicamente care expira în urmatoarele 30 de  
zile:');
```

```
dbms_output.put_line('-----  
---');
```

```
for record_medicament in (
```

```
    select nume_med, data_expirare_med
```

```
    from medicamente
```

```
    where data_expirare_med <= sysdate + 30
```

```
    order by data_expirare_med
```

```
)
```

```
loop
```

```
    dbms_output.put_line(
```

```
        'medicamentul: ' || record_medicament.nume_med ||
```

```
        ' - expira la: ' || to_char(record_medicament.data_expirare_med,  
'dd-mon-yyyy')
```

```
    );
```

```
end loop;
```

```
if sql%notfound then
```



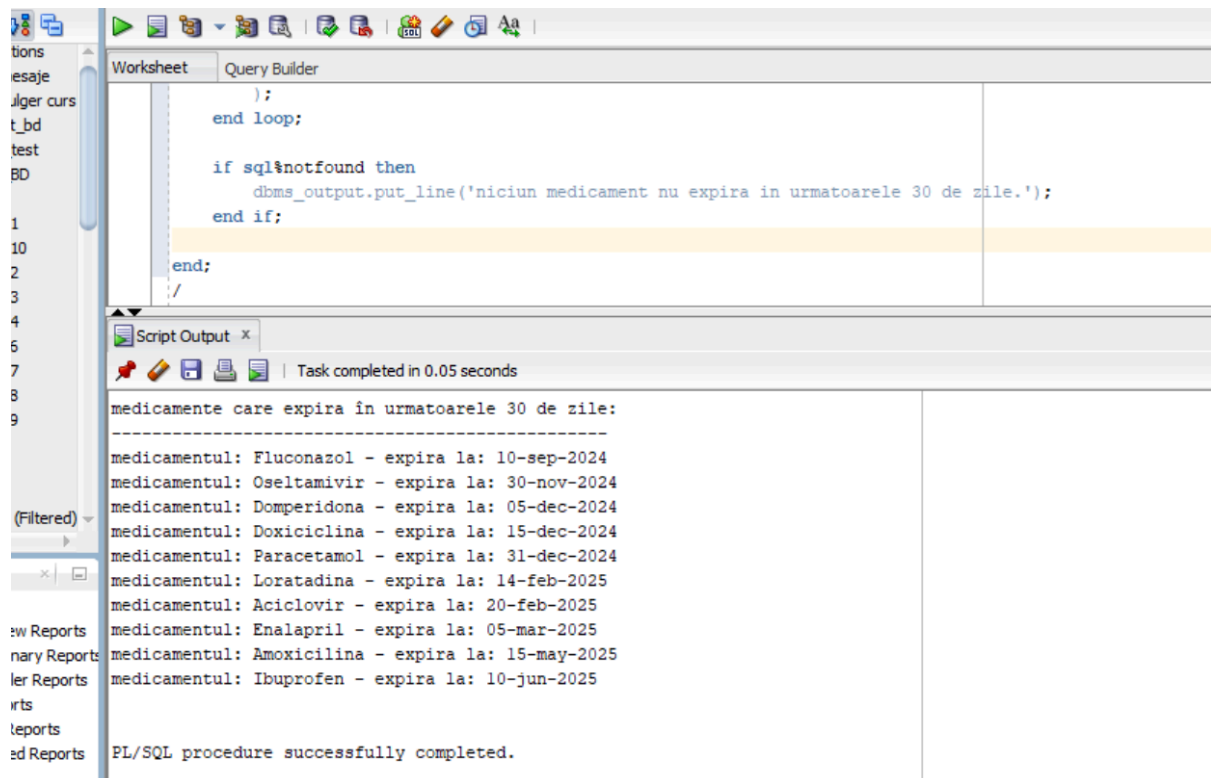
```

        dbms_output.put_line('niciun medicament nu expira in urmatoarele
30 de zile.');
```

end if;

```

end;
/
```



➤ Afișarea clienților cu rețete compensate.

```

begin
    dbms_output.put_line('clienti cu retete compensate:');
    dbms_output.put_line('-----');

    for rec_client in (
        select c.nume_client, c.prenume_client, c.email_client
        from retete r
        join clienti_farmacie c on r.id_client = c.id_client
```

```

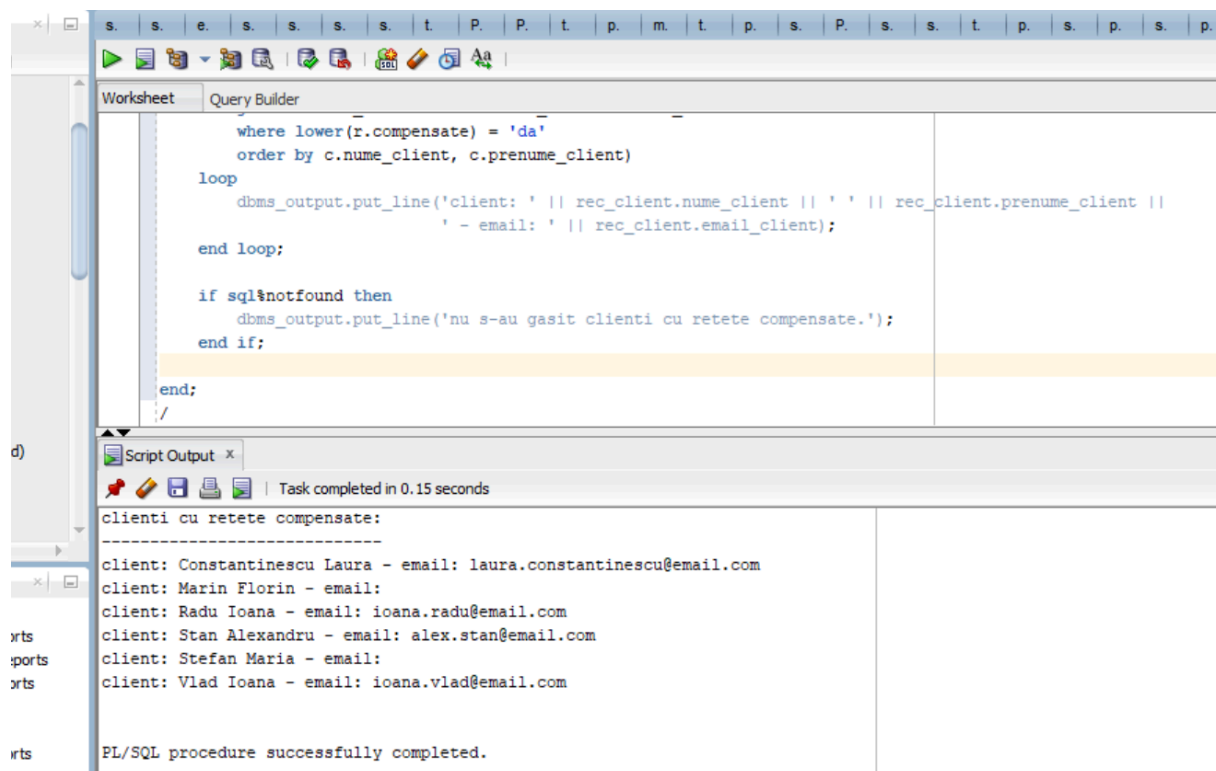
        where lower(r.compensate) = 'da'
        order by c.num_client, c.prename_client)
loop
    dbms_output.put_line('client: ' || rec_client.num_client || ' ' ||
rec_client.prename_client ||
        ' - email: ' || rec_client.email_client);
end loop;

if sql%notfound then
    dbms_output.put_line('nu s-au gasit clienti cu retete compensate.');
```

end if;

end;

/



The screenshot displays the Oracle SQL Developer environment. The top pane, titled 'Query Builder', contains the following PL/SQL code:

```

        where lower(r.compensate) = 'da'
        order by c.num_client, c.prename_client)
loop
    dbms_output.put_line('client: ' || rec_client.num_client || ' ' || rec_client.prename_client ||
        ' - email: ' || rec_client.email_client);
end loop;

if sql%notfound then
    dbms_output.put_line('nu s-au gasit clienti cu retete compensate.');
```

The bottom pane, titled 'Script Output', shows the results of the script execution. It indicates that the task was completed in 0.15 seconds and lists the output of the PL/SQL procedure:

```

clienti cu retete compensate:
-----
client: Constantinescu Laura - email: laura.constantinescu@email.com
client: Marin Florin - email:
client: Radu Ioana - email: ioana.radu@email.com
client: Stan Alexandru - email: alex.stan@email.com
client: Stefan Maria - email:
client: Vlad Ioana - email: ioana.vlad@email.com

PL/SQL procedure successfully completed.
```

➤ Istoricul vânzărilor pentru un client introdus de la tastatură.

declare

```

v_id_client number := &a;
v_total_vanzari_client number := 0;
begin
    dbms_output.put_line('vanzari pentru clientul cu id: ' || v_id_client);

    dbms_output.put_line('-----');

    for rec_vanzare in (
        select v.data_vanzare,
               m.nume_med,
               r.cantitate,
               r.pret_unitar
        from vanzari_medicamente v
        join rand_vanzari_med r on v.id_vanzare = r.id_vanzare
        join medicamente m on r.id_med = m.id_med
        where v.id_client = v_id_client
        order by v.data_vanzare
    )
    loop
        dbms_output.put_line(
            'data: ' || to_char(rec_vanzare.data_vanzare, 'dd-mon-yyyy') ||
            ' - medicament: ' || rec_vanzare.nume_med ||
            ' - cantitate: ' || rec_vanzare.cantitate ||
            ' - pret unitar: ' || rec_vanzare.pret_unitar
        );
        v_total_vanzari_client := v_total_vanzari_client + 1;
    end loop;

    if v_total_vanzari_client = 0 then
        dbms_output.put_line('nu s-au gasit vanzari pentru acest client.');
```

else

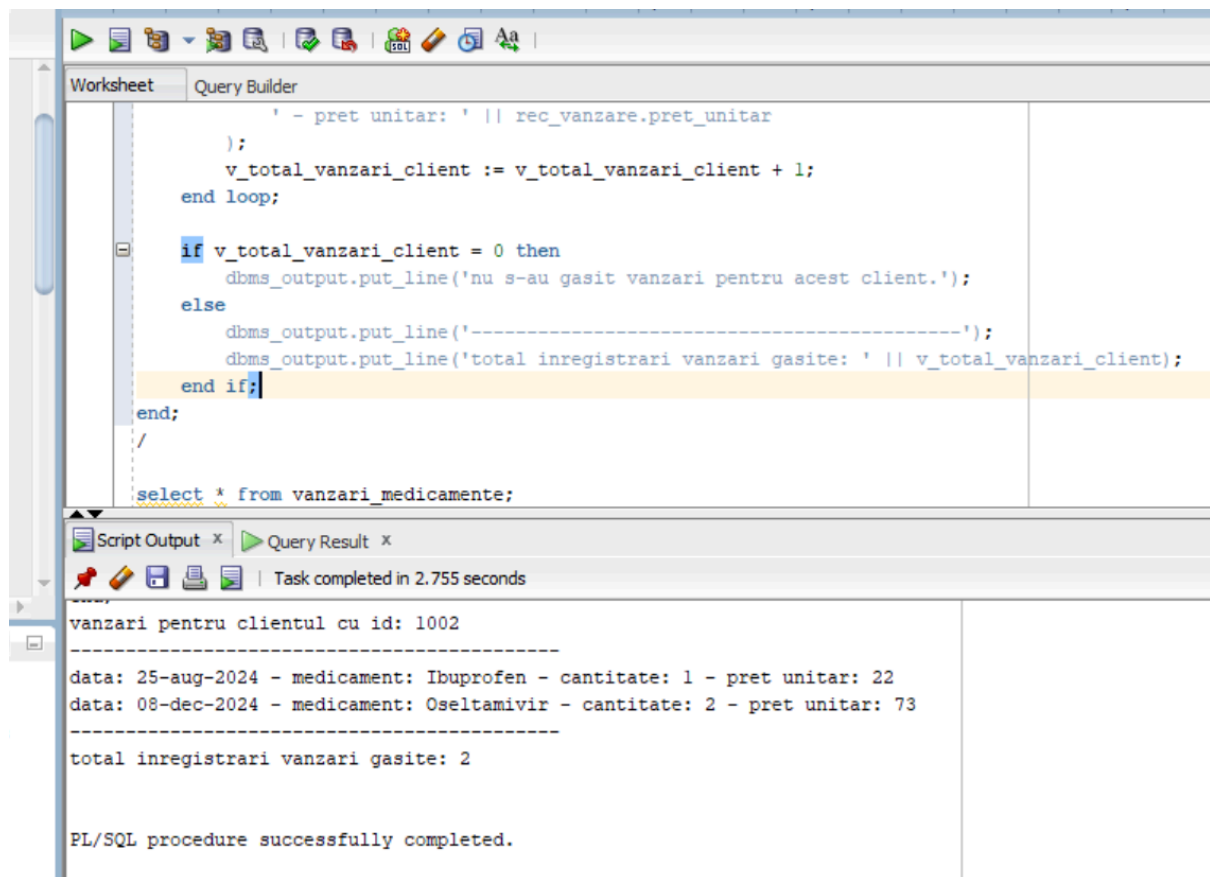
```

    dbms_output.put_line('-----');
```

```

        dbms_output.put_line('total inregistrari vanzari gasite: ' ||
v_total_vanzari_client);
    end if;
end;
/

```



The screenshot shows the Oracle SQL Developer interface. The top pane, titled 'Query Builder', contains a PL/SQL script. The bottom pane, titled 'Script Output', shows the results of the script's execution.

Script Code:

```

        - pret unitar: ' || rec_vanzare.pret_unitar
    );
    v_total_vanzari_client := v_total_vanzari_client + 1;
end loop;

if v_total_vanzari_client = 0 then
    dbms_output.put_line('nu s-au gasit vanzari pentru acest client.');
```

```

else
    dbms_output.put_line('-----');
    dbms_output.put_line('total inregistrari vanzari gasite: ' || v_total_vanzari_client);
end if;
end;
/

select * from vanzari_medicamente;

```

Script Output:

```

vanzari pentru clientul cu id: 1002

-----
data: 25-aug-2024 - medicament: Ibuprofen - cantitate: 1 - pret unitar: 22
data: 08-dec-2024 - medicament: Oseltamivir - cantitate: 2 - pret unitar: 73
-----
total inregistrari vanzari gasite: 2

PL/SQL procedure successfully completed.

```

- Calcularea salariului mediu al angajaților pe funcții și împărțire pe categorii de salarii.

declare

```
v_numar_functii number := 0;
```

```
v_categorie_salariu varchar2(20);
```

begin

```
dbms_output.put_line('salariile medii pe functii si categoria lor:');
```

```

dbms_output.put_line('-----')
;

for rec_salariu in (
    select f.denumire_functie,
           avg(a.salariu) as salariu_mediu
    from angajati_farmacie a
    join functii_farmacie f on a.id_functie = f.id_functie
    group by f.denumire_functie
    order by f.denumire_functie
)
loop
    case
        when rec_salariu.salariu_mediu < 3000 then
            v_categorie_salariu := 'mic';
            when rec_salariu.salariu_mediu >= 3000 and
rec_salariu.salariu_mediu < 6000 then
                v_categorie_salariu := 'mediu';
            when rec_salariu.salariu_mediu >= 6000 then
                v_categorie_salariu := 'mare';
            else
                v_categorie_salariu := 'nedefinit';
            end case;

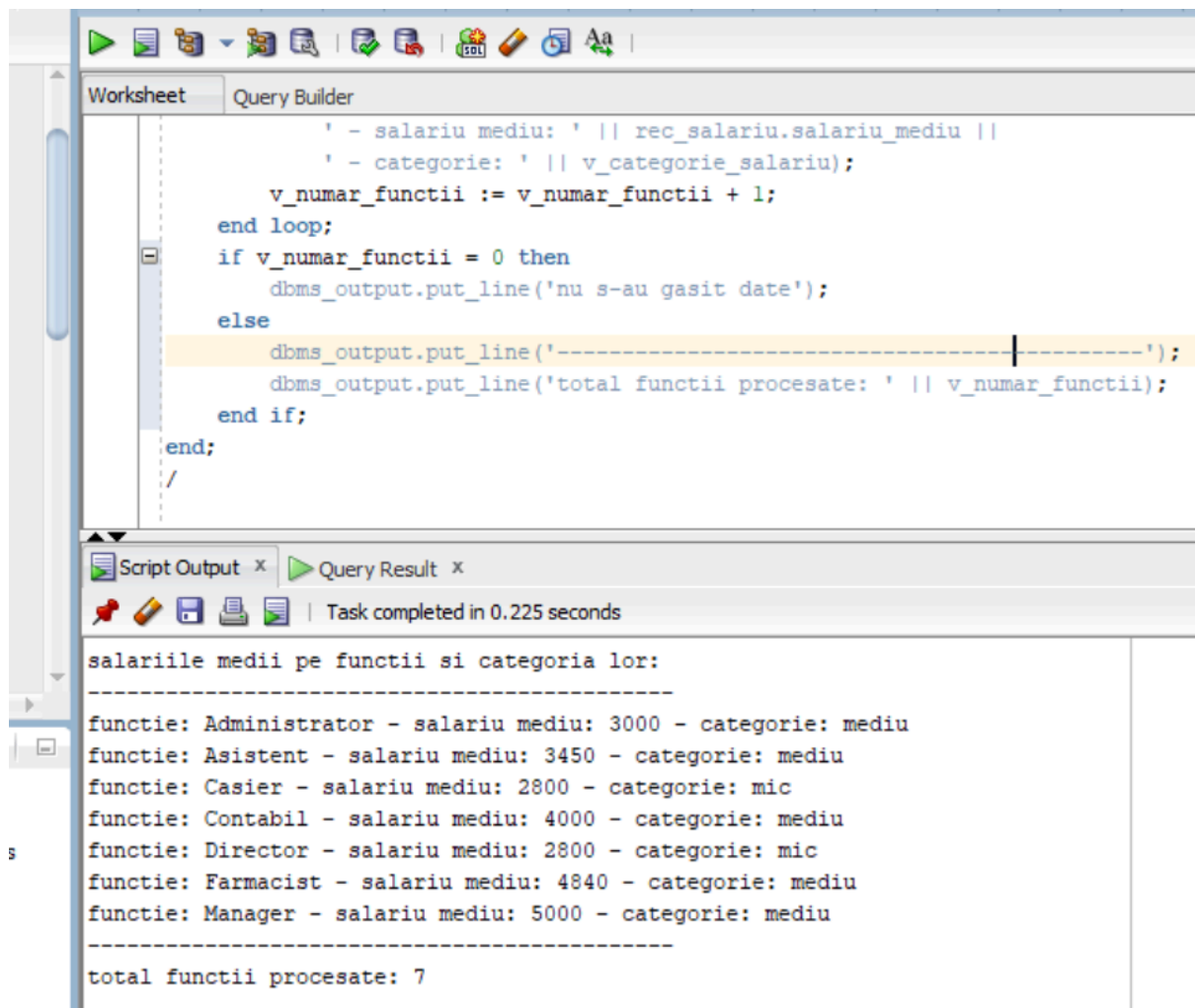
    dbms_output.put_line(
        'functie: ' || rec_salariu.denumire_functie ||
        ' - salariu mediu: ' || rec_salariu.salariu_mediu ||
        ' - categorie: ' || v_categorie_salariu);
    v_numar_functii := v_numar_functii + 1;
end loop;

if v_numar_functii = 0 then
    dbms_output.put_line('nu s-au gasit date');

```

else

```
dbms_output.put_line('-----');  
;  
    dbms_output.put_line('total functii procesate: ' || v_numar_functii);  
end if;  
end;  
/
```



The screenshot displays the Oracle SQL Developer environment. The top pane, titled 'Query Builder', contains a PL/SQL script. The script iterates through a list of job functions, calculates their average salary, and categorizes them. The bottom pane, titled 'Script Output', shows the execution results, including a header, a list of functions with their average salaries and categories, and a final summary line.

```
' - salariu mediu: ' || rec_salariu.salariu_mediu ||  
' - categorie: ' || v_categorie_salariu);  
v_numar_functii := v_numar_functii + 1;  
end loop;  
if v_numar_functii = 0 then  
    dbms_output.put_line('nu s-au gasit date');  
else  
    dbms_output.put_line('-----');  
    dbms_output.put_line('total functii procesate: ' || v_numar_functii);  
end if;  
end;  
/
```

Script Output x Query Result x
Task completed in 0.225 seconds

salariile medii pe functii si categoria lor:

```
-----  
functie: Administrator - salariu mediu: 3000 - categorie: mediu  
functie: Asistent - salariu mediu: 3450 - categorie: mediu  
functie: Casier - salariu mediu: 2800 - categorie: mic  
functie: Contabil - salariu mediu: 4000 - categorie: mediu  
functie: Director - salariu mediu: 2800 - categorie: mic  
functie: Farmacist - salariu mediu: 4840 - categorie: mediu  
functie: Manager - salariu mediu: 5000 - categorie: mediu  
-----  
total functii procesate: 7
```

b. Gestionarea cursorilor (impliciți și expliciți)

- Generarea unui raport de medicamente vândute într-o anumită perioadă.

```
set serveroutput on;
```

```
declare
```

```
cursor c_med_vandute(p_start date, p_end date) is
```

```
select m.nume_med, sum(rv.cantitate) as total
```

```
from rand_vanzari_med rv
```

```
join medicamente m on rv.id_med = m.id_med
```

```
join vanzari_medicamente v on rv.id_vanzare = v.id_vanzare
```

```
where v.data_vanzare between p_start and p_end
```

```
group by m.nume_med;
```

```
begin
```

```
for r in c_med_vandute(to_date('2024-01-01', 'yyyy-mm-dd'),  
to_date('2024-03-31', 'yyyy-mm-dd')) loop
```

```
dbms_output.put_line('medicament: ' || r.nume_med || ' - vandut: ' ||  
r.total);
```

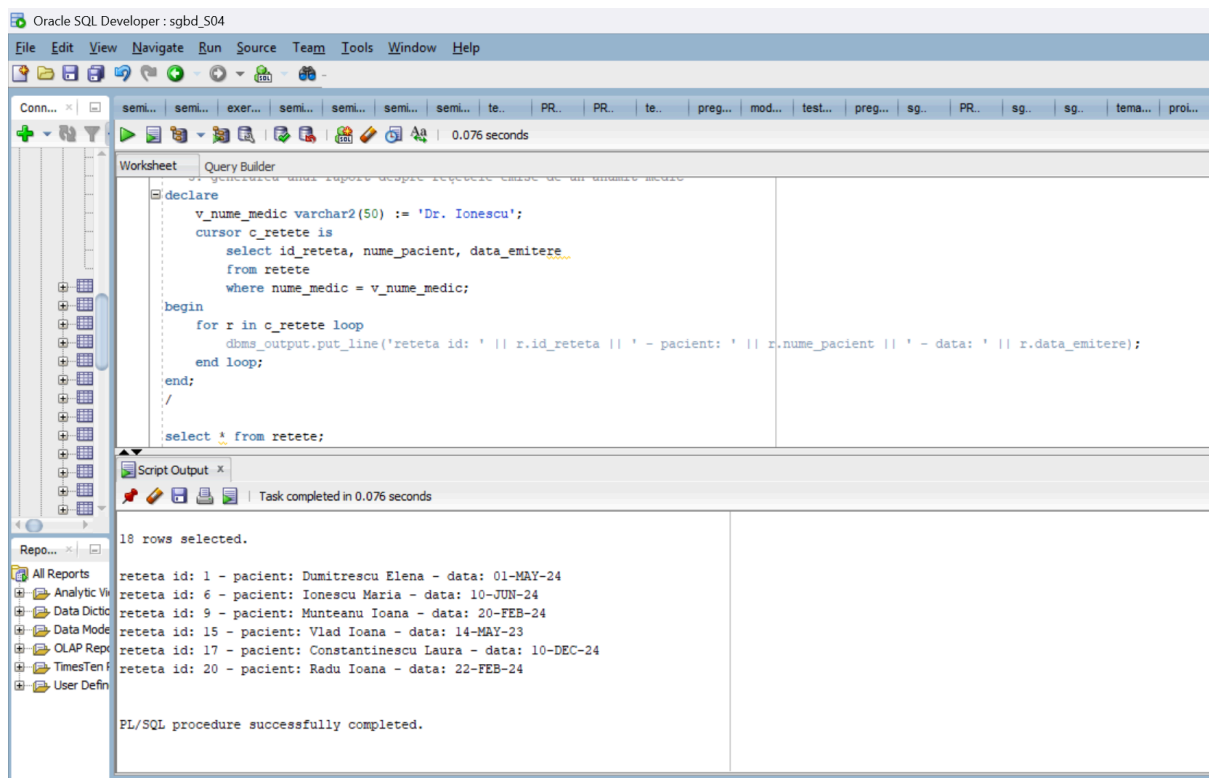
```
end loop;
```

```
end;
```

```
/
```


- Generarea unui raport despre rețetele emise de un anumit medic.

```
declare
v_num_medic varchar2(50) := 'Dr. Ionescu';
cursor c_retete is
select id_reteta, nume_pacient, data_emitere
from retete
where nume_medic = v_num_medic;
begin
for r in c_retete loop
dbms_output.put_line('reteta id: ' || r.id_reteta || ' - pacient: ' ||
r.nume_pacient || ' - data: ' || r.data_emitere);
end loop;
end;
/
```



- Identificarea clienților care au cumpărat același medicament de cel puțin 3 ori în ultimele 6 luni.

declare

cursor c_clienti_med is

```
select id_client, id_med, count(*) as nr_cumparari
from rand_vanzari_med rv
join vanzari_medicamente v on rv.id_vanzare = v.id_vanzare
where v.data_vanzare >= add_months(sysdate, -6)
group by id_client, id_med
having count(*) >= 1;
```

begin

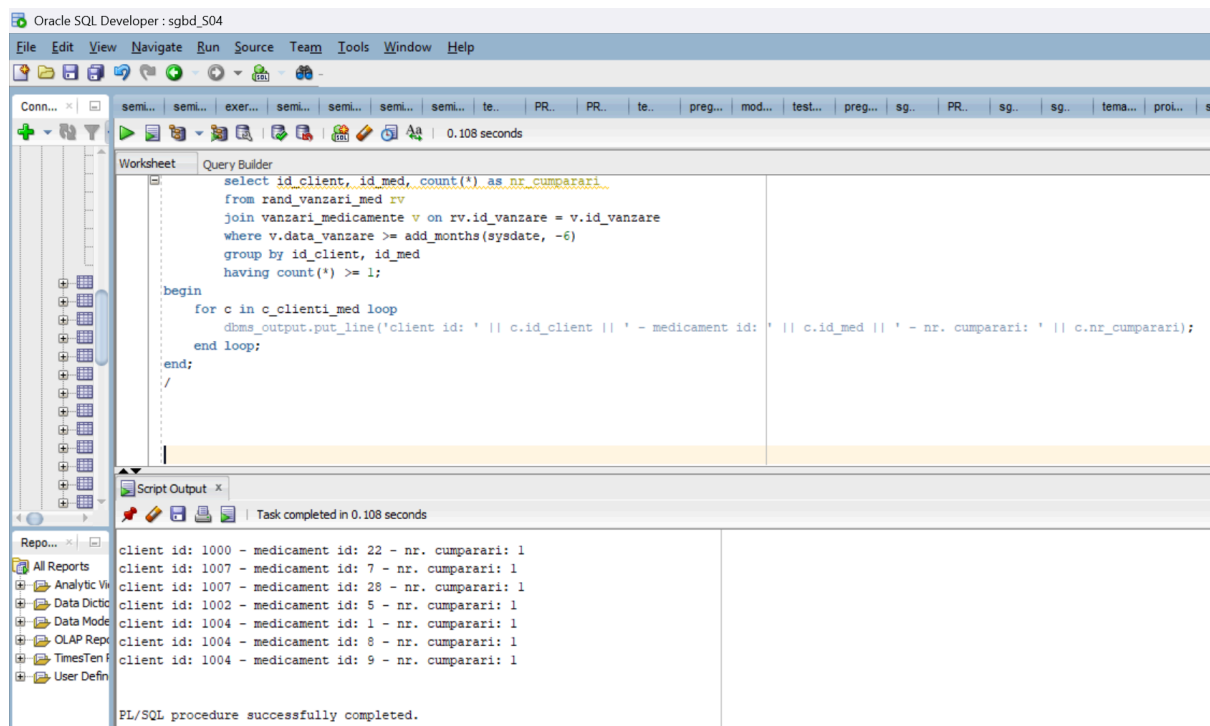
for c in c_clienti_med loop

```
    dbms_output.put_line('client id: ' || c.id_client || ' - medicament id: ' ||
c.id_med || ' - nr. cumparari: ' || c.nr_cumparari);
```

end loop;

end;

/



- Generarea unei liste de angajați care au salarii sub media departamentului lor.

declare

cursor c_angajati is

select a.id_angajat, a.numa_angajat, a.salariu, a.id_functie

from angajati_farmacie a

where a.salariu < (select avg(salariu) from angajati_farmacie where
id_functie = a.id_functie);

begin

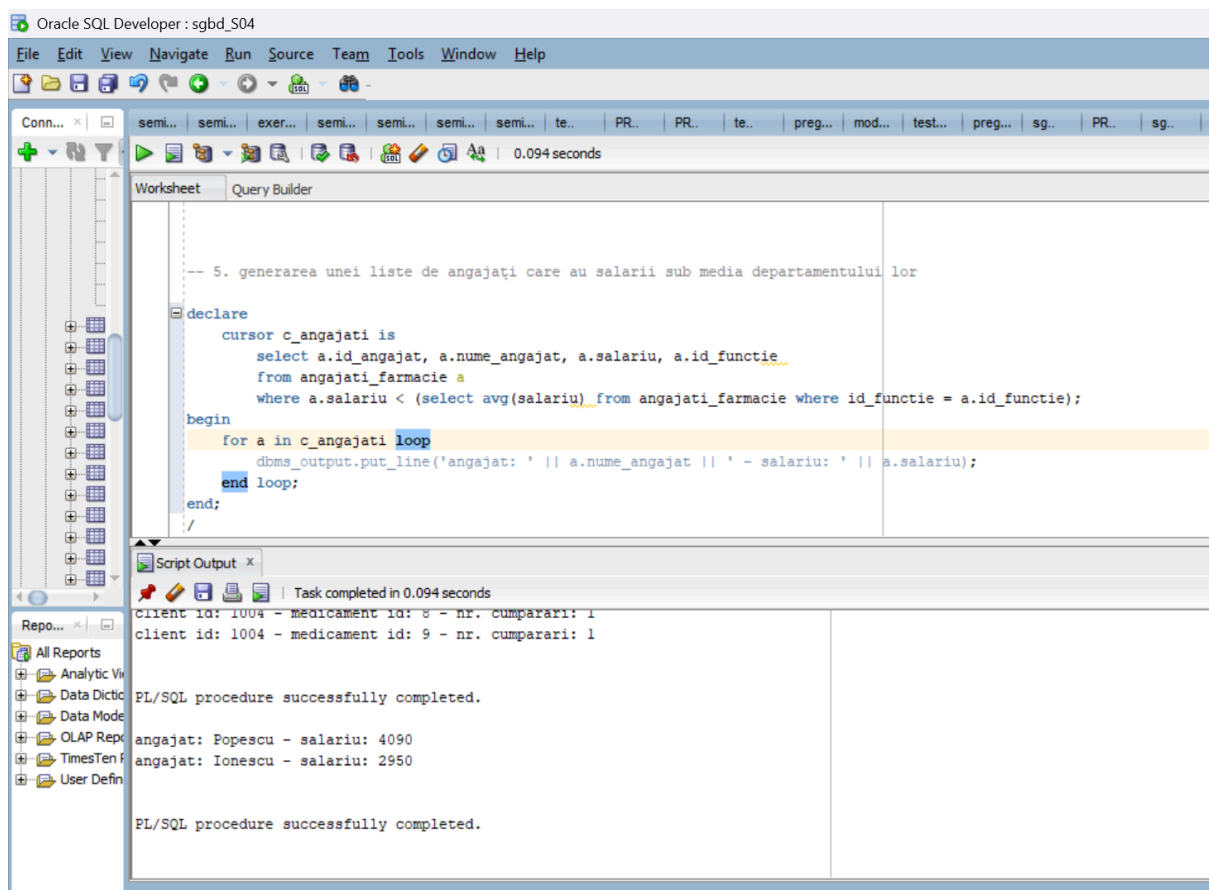
for a in c_angajati loop

dbms_output.put_line('angajat: ' || a.numa_angajat || ' - salariu: ' ||
a.salariu);

end loop;

end;

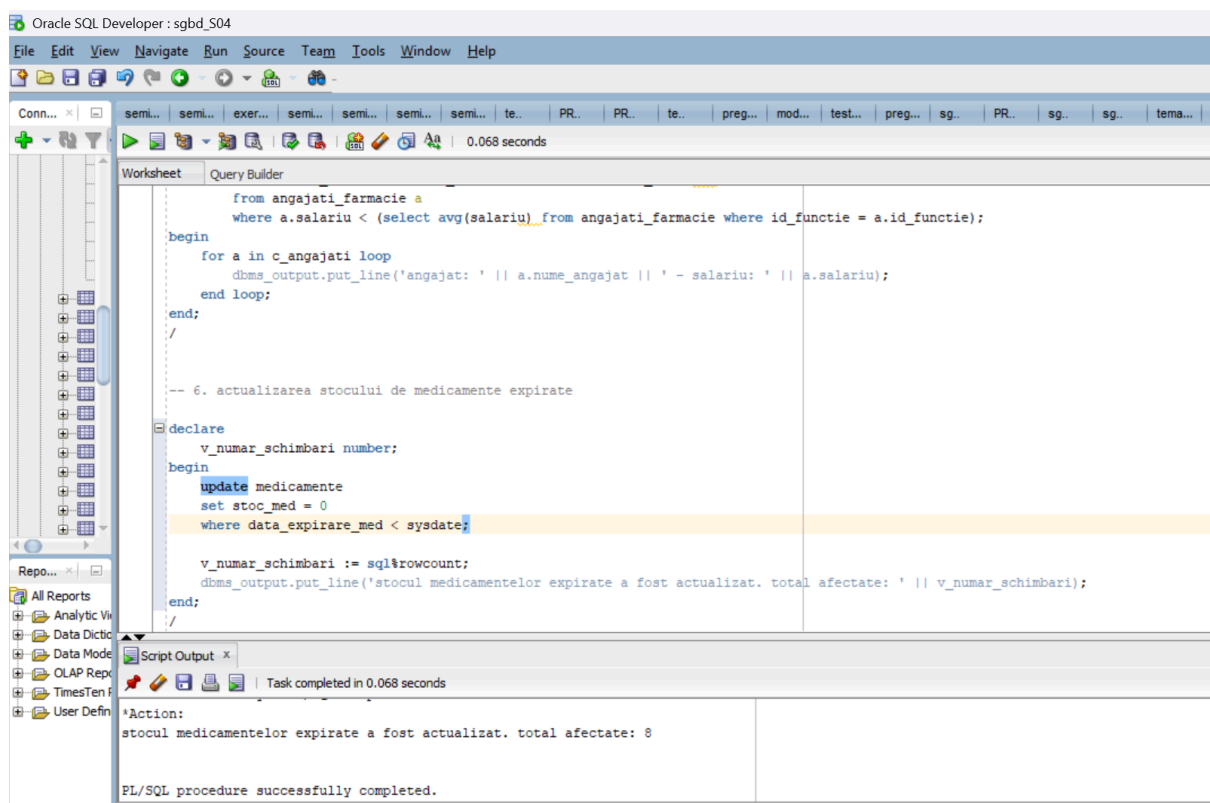
/



➤ Actualizarea stocului de medicamente expirate.

```
declare
    v_numar_schimbari number;
begin
    update medicamente
    set stoc_med = 0
    where data_expirare_med < sysdate;

    v_numar_schimbari := sql%rowcount;
    dbms_output.put_line('stocul medicamentelor expirate a fost actualizat.
total afectate: ' || v_numar_schimbari);
end;
/
```



c. Tratarea excepțiilor (implicite și explicite)

- Verificarea stocului medicamentelor: avertizare dacă stocul e critic.

```
set serveroutput on
```

```
declare
```

```
  v_num_med varchar2(50) := lower('&num_med');
  v_stoc medicamente.stoc_med%type;
```

```
  v_gasit boolean := false;
```

```
  ex_stoc_critic exception;
```

```
  ex_nu_exista exception;
```

```
begin
```

```
  select stoc_med
```

```
  into v_stoc
```

```
  from medicamente
```

```
  where lower(num_med) = v_num_med;
```

```
  v_gasit := true;
```

```
  if v_stoc < 5 then
```

```
    raise ex_stoc_critic;
```

```
  else
```

```
    dbms_output.put_line('stocul este suficient pentru medicamentul ' ||
```

```
v_num_med || ': ' || v_stoc || ' bucati');
```

```
  end if;
```

```
exception
```

```
  when ex_stoc_critic then
```

```
    dbms_output.put_line('atentie: stoc critic pentru medicamentul ' ||
```

```
v_num_med || ': ' || v_stoc || ' bucati');
```

```
  when ex_nu_exista then
```

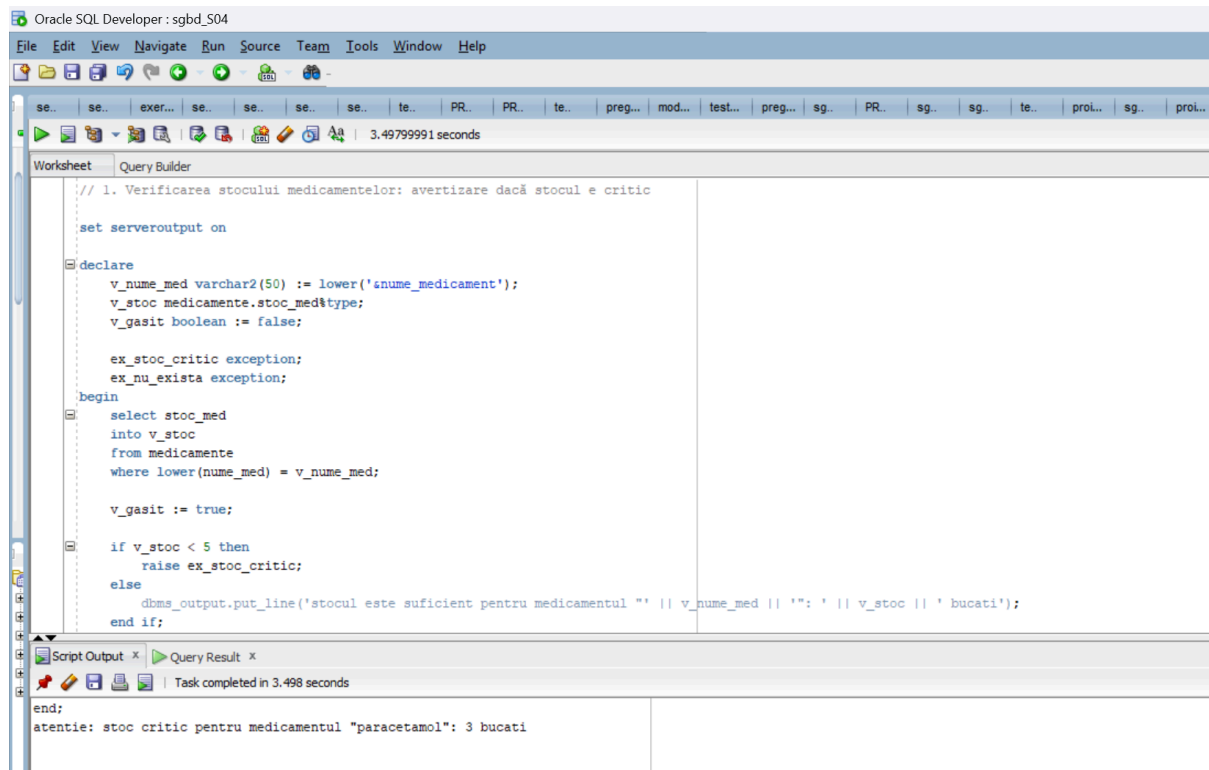
```

        dbms_output.put_line('medicamentul "' || v_num_med || '" nu exista
in baza de date.');
```

```

end;

/
```



- Calcularea salariului mediu al angajaților care ocupă o anumită funcție.

```

declare
```

```

    v_denumire_functie varchar2(50) := '&functie';
    v_id_functie functii_farmacie.id_functie%type;
    v_salariu_total number;
    v_nr_angajati number;
    v_salariu_mediu number;
    b_functie_gasita boolean := true;
    ex_fara_angajati_pentru_functie exception;
    pragma exception_init(ex_fara_angajati_pentru_functie, -20001);
```

```

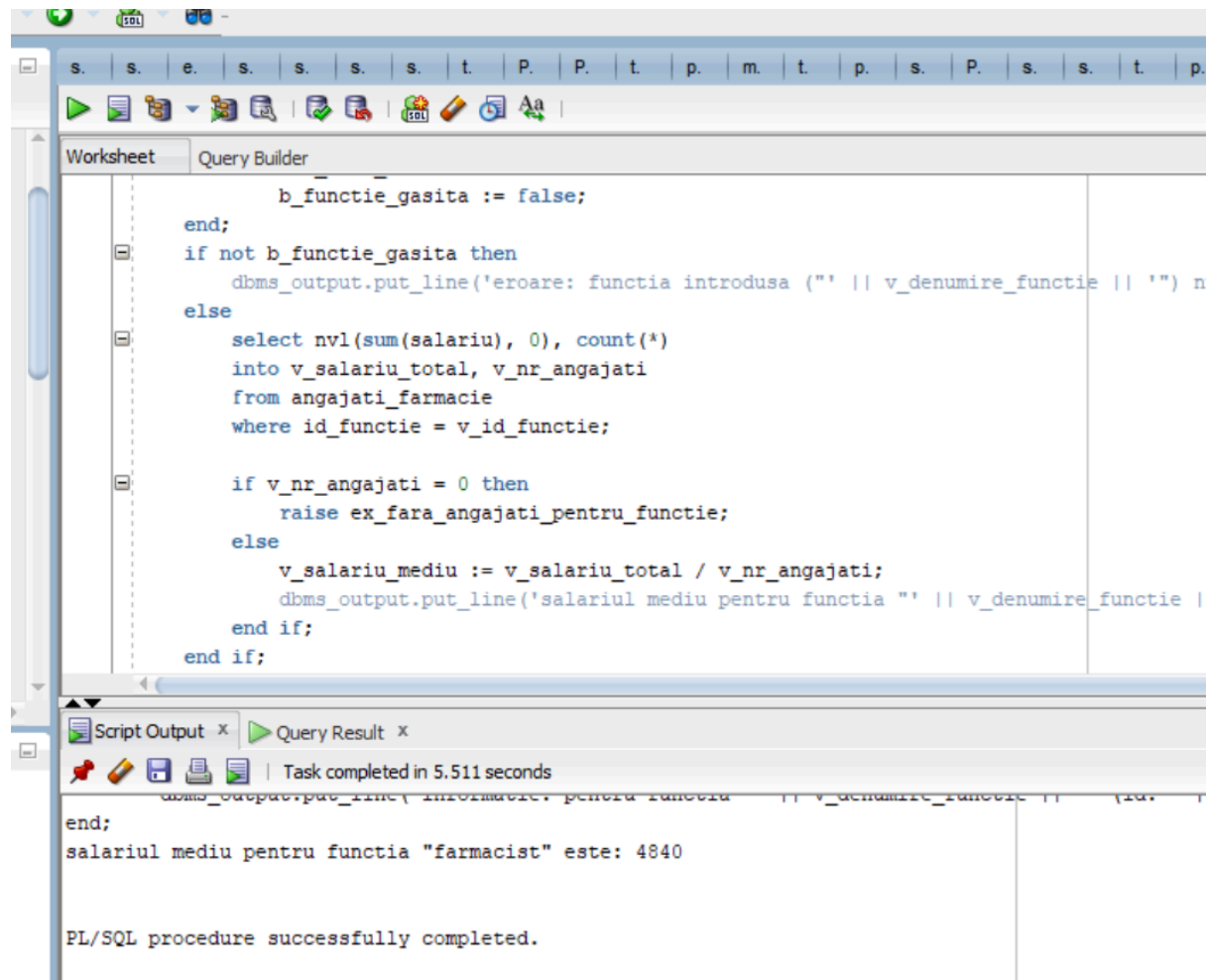
begin
    begin
        select id_functie
        into v_id_functie
        from functii_farmacie
                                where    upper(trim(denumire_functie))    =
upper(trim(v_denumire_functie));
    exception
        when no_data_found then
            b_functie_gasita := false;
    end;
    if not b_functie_gasita then
        dbms_output.put_line('eroare: functia introdusa (' || v_denumire_functie || ') nu exista!');
    else
        select nvl(sum(salariu), 0), count(*)
        into v_salariu_total, v_nr_angajati
        from angajati_farmacie
        where id_functie = v_id_functie;

        if v_nr_angajati = 0 then
            raise ex_fara_angajati_pentru_functie;
        else
            v_salariu_mediu := v_salariu_total / v_nr_angajati;
            dbms_output.put_line('salariul mediu pentru functia ' || v_denumire_functie || ' este: ' || round(v_salariu_mediu, 2));
        end if;
    end if;

exception
    when ex_fara_angajati_pentru_functie then
        dbms_output.put_line('informatie: pentru functia ' || v_denumire_functie || ' (id: ' || v_id_functie || ') nu exista angajati inregistrati.');
```

end;

/



The screenshot displays the Oracle SQL Developer environment. The top toolbar includes icons for running queries, saving, and other standard database operations. The main window is titled 'Query Builder' and contains a PL/SQL procedure. The procedure logic is as follows:

```
        b_functie_gasita := false;
    end;
    if not b_functie_gasita then
        dbms_output.put_line('eroare: functia introdusa (' || v_denumire_functie || ') n
    else
        select nvl(sum(salariu), 0), count(*)
        into v_salariu_total, v_nr_angajati
        from angajati_farmacie
        where id_functie = v_id_functie;

        if v_nr_angajati = 0 then
            raise ex_fara_angajati_pentru_functie;
        else
            v_salariu_mediu := v_salariu_total / v_nr_angajati;
            dbms_output.put_line('salariul mediu pentru functia ' || v_denumire_functie |
        end if;
    end if;
```

Below the code editor, the 'Script Output' window shows the results of the procedure execution. It indicates that the task was completed in 5.511 seconds and displays the output of the `dbms_output.put_line` statement:

```
salariul mediu pentru functia "farmacist" este: 4840
```

The final message in the output window is 'PL/SQL procedure successfully completed.'

➤ Total vânzări pentru un client (cu excepție dacă nu există vânzări)

declare

v_id_client number := &id_client;

v_total number := 0;

cursor c_vanzari is

select r.cantitate * r.pret_unitar as total

from vanzari_medicamente v

join rand_vanzari_med r on v.id_vanzare = r.id_vanzare

where v.id_client = v_id_client;


```

v_partial number;

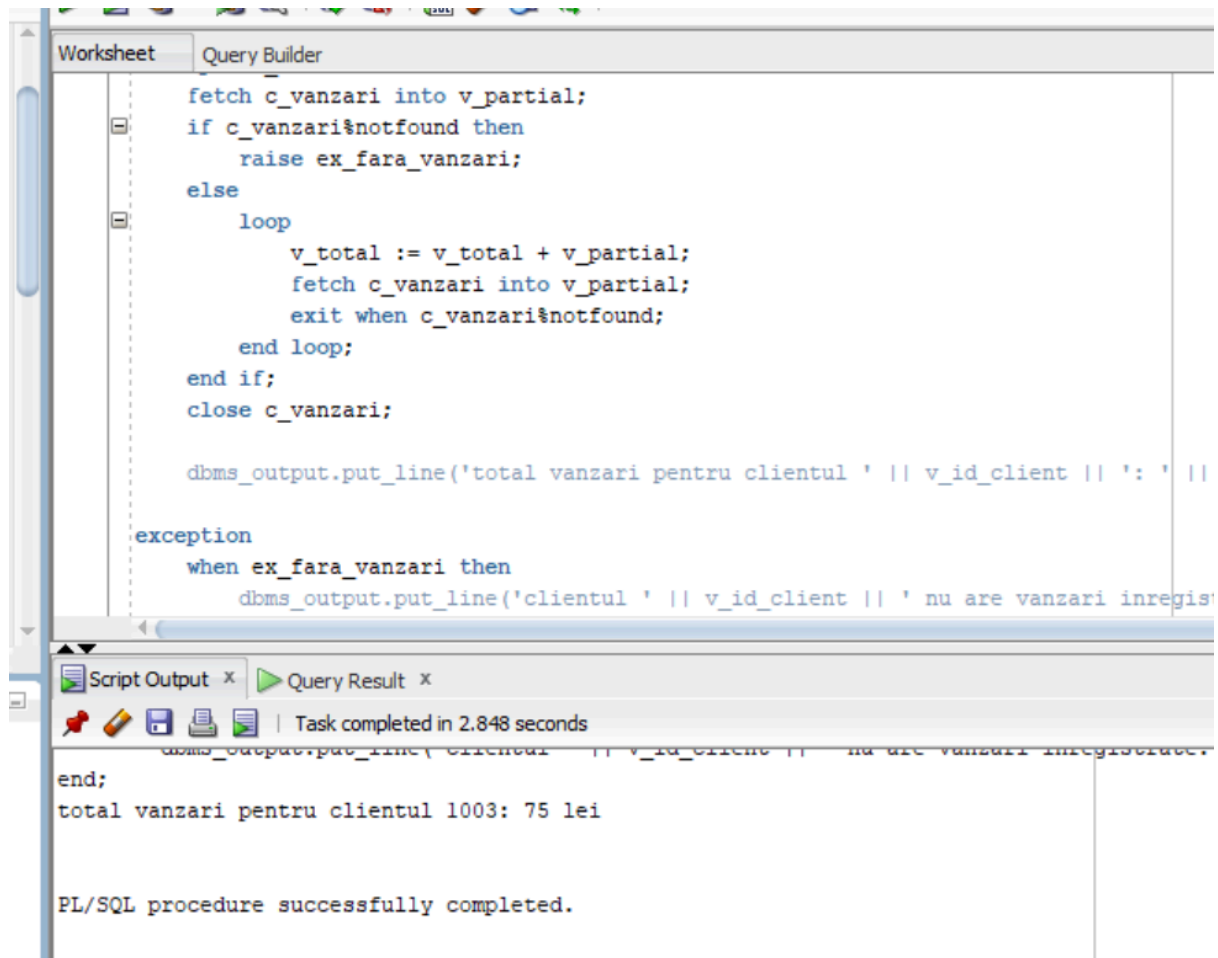
ex_fara_vanzari exception;
begin
    open c_vanzari;
    fetch c_vanzari into v_partial;
    if c_vanzari%notfound then
        raise ex_fara_vanzari;
    else
        loop
            v_total := v_total + v_partial;
            fetch c_vanzari into v_partial;
            exit when c_vanzari%notfound;
        end loop;
    end if;
    close c_vanzari;

    dbms_output.put_line('total vanzari pentru clientul ' || v_id_client || ':' ||
v_total || ' lei');

exception
    when ex_fara_vanzari then
        dbms_output.put_line('clientul ' || v_id_client || ' nu are vanzari
inregistrate.');
```

end;

/



The screenshot displays the Oracle SQL Developer interface. The top pane, titled 'Query Builder', contains a PL/SQL script. The script defines a cursor to fetch invoice data, calculates a total, and includes an exception handler for missing data. The bottom pane, titled 'Script Output', shows the execution results, including the total for client 1003 and a confirmation that the procedure completed successfully.

```
fetch c_vanzari into v_partial;
if c_vanzari%notfound then
    raise ex_fara_vanzari;
else
    loop
        v_total := v_total + v_partial;
        fetch c_vanzari into v_partial;
        exit when c_vanzari%notfound;
    end loop;
end if;
close c_vanzari;

dbms_output.put_line('total vanzari pentru clientul ' || v_id_client || ': ' ||

exception
when ex_fara_vanzari then
    dbms_output.put_line('clientul ' || v_id_client || ' nu are vanzari inregis'
```

Task completed in 2.848 seconds

total vanzari pentru clientul 1003: 75 lei

PL/SQL procedure successfully completed.

➤ Verificare furnizori cu date lipsa.

declare

cursor c_furnizori is

select nume_furnizor, telefon_furnizor, email_furnizor
from furnizori_med;

v_nume furnizori_med.nume_furnizor%type;

v_tel furnizori_med.telefon_furnizor%type;

v_email furnizori_med.email_furnizor%type;

ex_date_incomplete exception;

begin

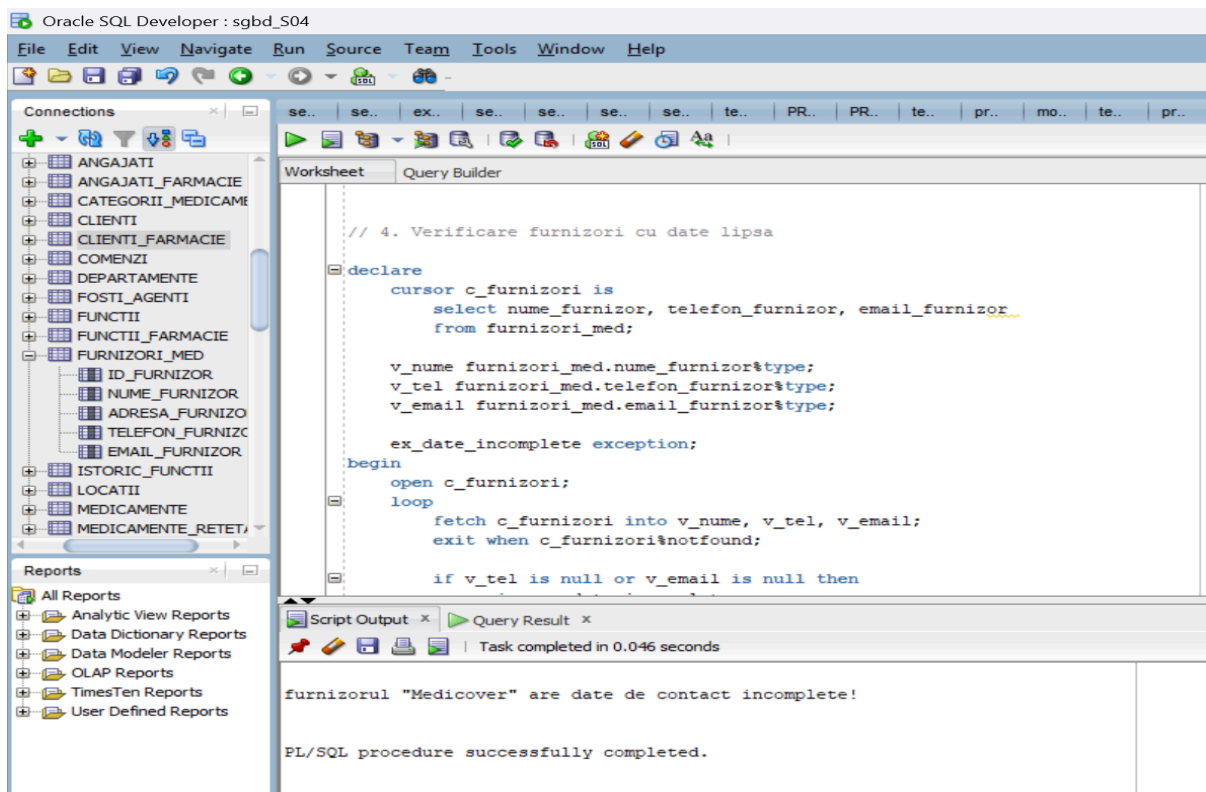
```

open c_furnizori;
loop
    fetch c_furnizori into v_num, v_tel, v_email;
    exit when c_furnizori%notfound;

    if v_tel is null or v_email is null then
        raise ex_date_incomplete;
    else
        dbms_output.put_line('furnizor: ' || v_num || ' - tel: ' || v_tel || ' -
email: ' || v_email);
    end if;
end loop;
close c_furnizori;

exception
    when ex_date_incomplete then
        dbms_output.put_line('furnizorul "' || v_num || '" are date de contact
incomplete!');
end; /

```



d. Funcții, proceduri, pachete

➤ Procedură pentru actualizarea stocului de medicamente.

```
create or replace procedure p_actualizeaza_stoc_medicament (
    p_id_med in medicamente.id_med%type,
    p_cantitate_adaugata in number
) is
    ex_cantitate_invalida exception;
    pragma exception_init(ex_cantitate_invalida, -20001);
    v_id medicamente.id_med%type;
begin
    if p_cantitate_adaugata < 0 then
        raise ex_cantitate_invalida;
    end if;

    select id_med
    into v_id
    from medicamente
    where id_med = p_id_med;

    update medicamente
    set stoc_med = stoc_med + p_cantitate_adaugata
    where id_med = p_id_med;

    dbms_output.put_line('stoc actualizat pentru medicamentul cu id ' ||
p_id_med || '. cantitate adaugata: ' || p_cantitate_adaugata);

exception
    when no_data_found then
        dbms_output.put_line('eroare (p_actualizeaza_stoc_medicament):
medicamentul cu id ' || p_id_med || ' nu exista.');
```

```
    when ex_cantitate_invalida then
```

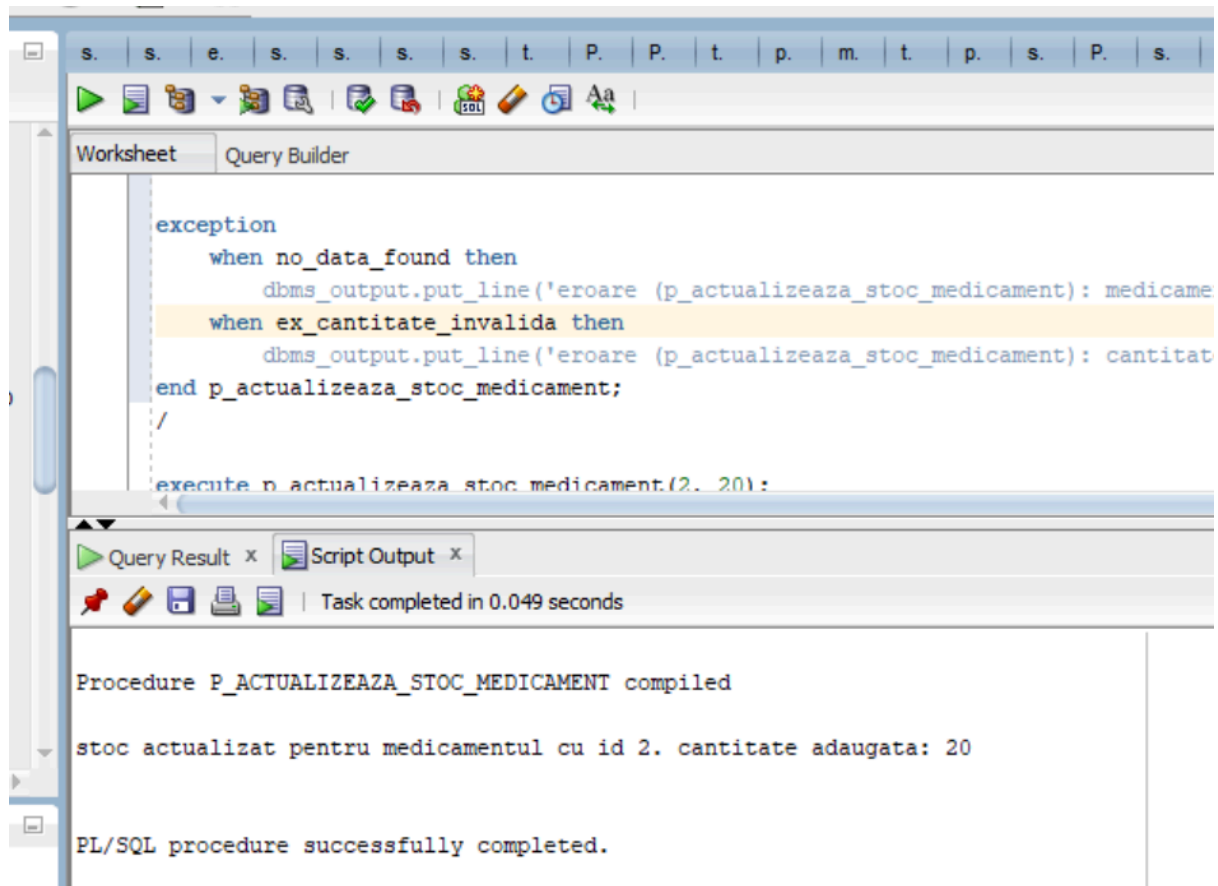
```

        dbms_output.put_line('eroare (p_actualizeaza_stoc_medicament):
cantitatea adaugata (' || p_cantitate_adaugata || ') nu poate fi negativa.');
```

```

end p_actualizeaza_stoc_medicament;
/

execute p_actualizeaza_stoc_medicament(2, 20);
```



➤ Procedură pentru modificarea prețului unui medicament.

```

create or replace procedure p_modifica_pret_medicament (
  p_id_med in medicamente.id_med%type,
  p_procent_modificare in number
) is
  v_pret_curent medicamente.pret_med%type;
  v_pret_nou medicamente.pret_med%type;
  ex_pret_invalid exception;
```

```

pragma exception_init(ex_pret_invalid, -20002);
begin
    select pret_med
    into v_pret_curent
    from medicamente
    where id_med = p_id_med;
    v_pret_nou := v_pret_curent * (1 + p_procent_modificare / 100);
    if v_pret_nou <= 0 then
        raise ex_pret_invalid;
    end if;

    update medicamente
    set pret_med = round(v_pret_nou, 2)
    where id_med = p_id_med;

    dbms_output.put_line('pret actualizat pentru medicamentul id ' ||
p_id_med || '. noul pret: ' || round(v_pret_nou, 2));

exception
    when no_data_found then
        dbms_output.put_line('eroare (p_modifica_pret_medicament):
medicamentul cu id ' || p_id_med || ' nu exista.');
```

```

    when ex_pret_invalid then
        dbms_output.put_line('eroare (p_modifica_pret_medicament):
modificarea procentuala (' || p_procent_modificare || '%) pentru
medicamentul id ' || p_id_med || ' face ca pretul sa fie invalid (<=0). pret
calculat: ' || v_pret_nou);

end p_modifica_pret_medicament;
/

execute p_modifica_pret_medicament (4, 30);
```

The screenshot shows the Oracle SQL Developer interface. At the top, there are two tabs: 'Worksheet' and 'Query Builder'. The 'Worksheet' tab is active, displaying a PL/SQL procedure named `p_modifica_pret_medicament`. The procedure's code is as follows:

```
dbms_output.put_line('pret actualizat pentru medicamen  
exception  
  when no_data_found then  
    dbms_output.put_line('eroare (p_modifica_pret_medi  
  when ex_pret_invalid then  
    dbms_output.put_line('eroare (p_modifica_pret_medi  
end p_modifica_pret_medicament;  
/  
execute p_modifica_pret_medicament (4, 30);
```

Below the code editor, there is a status bar with icons for a pin, a pencil, a save icon, a print icon, and a refresh icon. To the right of these icons, it says 'Task completed in 0.059 seconds'. Below the status bar, there are two tabs: 'Query Result' and 'Script Output'. The 'Script Output' tab is active, displaying the following text:

```
Procedure P_MODIFICA_PRET_MEDICAMENT compiled  
pret actualizat pentru medicamentul id 4. noul pret: 33  
PL/SQL procedure successfully completed.
```

➤ Procedură pentru afișarea informațiilor despre un medicament.

```
create or replace procedure p_afiseaza_info_medicament (  
  p_id_med in medicamente.id_med%type) is  
  rec_medicament medicamente%rowtype;  
begin  
  select * into rec_medicament from medicamente
```

```

where id_med = p_id_med;
        dbms_output.put_line('informatii medicament cu id: ' ||
rec_medicament.id_med);
        dbms_output.put_line(' nume: ' || rec_medicament.nume_med);
        dbms_output.put_line('          producator:      ' ||
rec_medicament.producator_med);
        dbms_output.put_line(' pret: ' || rec_medicament.pret_med);
        dbms_output.put_line(' stoc: ' || rec_medicament.stoc_med);
        dbms_output.put_line('          data expirare:      ' ||
to_char(rec_medicament.data_expirare_med, 'dd-mon-yyyy'));
exception
        when no_data_found then dbms_output.put_line('eroare
(p_afiseaza_info_medicament): medicamentul cu id ' || p_id_med || ' nu
exista.');
```

```

end p_afiseaza_info_medicament;/
execute p_afiseaza_info_medicament(5);
```

```

exception
    when no_data_found then
        dbms_output.put_line('eroare (p_afiseaza_info_medicament): 1
    end p_afiseaza_info_medicament;
/
execute p_afiseaza_info_medicament(5);
```

Query Result x Script Output x

Task completed in 0.064 seconds

```

PL/SQL procedure successfully completed.

Procedure P_AFISEAZA_INFO_MEDICAMENT compiled

informatii medicament cu id: 5
  nume: Oseltamivir
producator: Novartis
  pret: 50
  stoc: 3
  data expirare: 30-nov-2024

PL/SQL procedure successfully completed.
```


➤ Funcție pentru calcularea stocului.

```
create or replace function f_calculeaza_valoare_stoc_medicament
(p_id_med in medicamente.id_med%type) return number is
    v_valoare_stoc number;
    v_stoc medicamente.stoc_med%type;
    v_pret medicamente.pret_med%type;
begin
    select stoc_med, pret_med
    into v_stoc, v_pret
    from medicamente
    where id_med = p_id_med;

    v_valoare_stoc := v_stoc * v_pret;
    return v_valoare_stoc;

exception
    when no_data_found then
        dbms_output.put_line('eroare
(f_calculeaza_valoare_stoc_medicament): medicamentul cu id ' ||
p_id_med || ' nu exista.');
```

```
end f_calculeaza_valoare_stoc_medicament;
/

begin
    dbms_output.put_line('Valoarea stocului este: ' ||
f_calculeaza_valoare_stoc_medicament(1));
exception
    when no_data_found then
        dbms_output.put_line('Medicamentul specificat nu a fost gasit.');
```

```
end;
/
```

The screenshot displays the Oracle SQL Developer environment. The top toolbar shows various icons, and the top right corner indicates a duration of 0.149 seconds. The main window is titled 'Query Builder' and contains the following PL/SQL code:

```
when no_data_found then
    dbms_output.put_line('eroare (f_calculeaza_v
end f_calculeaza_valoare_stoc_medicament;
/

begin
    dbms_output.put_line('Valoarea stocului este: '
exception
    when no_data_found then
        dbms_output.put_line('Medicamentul specifica
end;
/
```

Below the code editor, the 'Script Output' window is open, showing the results of the execution:

```
PL/SQL procedure successfully completed.

Function F_CALCULEAZA_VALOARE_STOC_MEDICAMENT compiled

Valoarea stocului este: 5000

PL/SQL procedure successfully completed.
```

➤ Funcție pentru verificarea valabilității medicamentelor.

```
create or replace function f_verifica_expirare_medicament (
    p_id_med in medicamente.id_med%type
) return boolean is
```

```

    v_data_expirare medicamente.data_expirare_med%type;
begin
    select data_expirare_med
    into v_data_expirare
    from medicamente
    where id_med = p_id_med;

    if v_data_expirare < trunc(sysdate) then
        return true;
    else
        return false;
    end if;
exception
    when no_data_found then
        dbms_output.put_line('eroare (f_verifica_expirare_medicament):
medicamentul cu id ' || p_id_med || ' nu exista.');
```

end f_verifica_expirare_medicament;

/

```

set serveroutput on;

declare
    v_id_medicament_test medicamente.id_med%type;
    v_este_expirat    boolean;
begin
    v_id_medicament_test := 4;

    dbms_output.put_line('Se verifica medicamentul cu ID: ' ||
v_id_medicament_test);

                                v_este_expirat :=
f_verifica_expirare_medicament(v_id_medicament_test);
```

```

if v_este_expirat then
    dbms_output.put_line('Rezultat: Medicamentul ESTE expirat.');
```

else

```

    dbms_output.put_line('Rezultat: Medicamentul NU este expirat.');
```

end if;

exception

```

    when no_data_found then
        dbms_output.put_line('EROARE: Medicamentul cu ID ' ||
v_id_medicament_test || ' nu a fost gasit. Verificarea expirarii nu a putut fi
efectuata.');
```

end;

/

```

exception
    when no_data_found then
        dbms_output.put_line('EROARE: Medicamentul cu ID
end;
/

```

Query Result x Script Output x

Task completed in 0.051 seconds

PL/SQL procedure successfully completed.

Function F_VERIFICA_EXPIRARE_MEDICAMENT compiled

Se verifica medicamentul cu ID: 4

Rezultat: Medicamentul NU este expirat.

PL/SQL procedure successfully completed.

➤ Funcție pentru a calcula valoarea vânzărilor către un client.

```

create or replace function f_valoare_totala_vanzari_client (
    p_id_client in clienti_farmacie.id_client%type,
    p_data_inceput in date,
    p_data_sfarsit in date
) return number is
    v_total_valoare number := 0;
    v_id clienti_farmacie.id_client%type;
begin
    select id_client
    into v_id
    from clienti_farmacie
    where id_client = p_id_client;

    if p_data_inceput > p_data_sfarsit then
        dbms_output.put_line('eroare: data de inceput (' ||
            to_char(p_data_inceput, 'dd-mm-yyyy') ||
            ') este dupa data de sfarsit (' ||
            to_char(p_data_sfarsit, 'dd-mm-yyyy') || ')');
        return 0;
    end if;

    select nvl(sum(rvm.cantitate * rvm.pret_unitar), 0)
    into v_total_valoare
    from vanzari_medicamente vm
    join rand_vanzari_med rvm on vm.id_vanzare = rvm.id_vanzare
    where vm.id_client = p_id_client
        and vm.data_vanzare between p_data_inceput and p_data_sfarsit;

    return v_total_valoare;

exception
    when no_data_found then

```

```
        dbms_output.put_line('eroare (f_valoare_totala_vanzari_client):  
clientul cu id ' || p_id_client || ' nu exista.');
```

end f_valoare_totala_vanzari_client;

/

set serveroutput on;

declare

```
    v_id_client_test  clienti_farmacie.id_client%type;  
    v_data_start_test date;  
    v_data_end_test   date;  
    v_total_vanzari   number;
```

begin

```
    v_id_client_test := 1004;  
    v_data_start_test := trunc(sysdate) - 30;  
    v_data_end_test   := trunc(sysdate);
```

```
        dbms_output.put_line('Se calculeaza valoarea vanzarilor pentru client  
ID: ' || v_id_client_test ||  
        ' intre ' || to_char(v_data_start_test, 'dd-mm-yyyy') ||  
        ' si ' || to_char(v_data_end_test, 'dd-mm-yyyy'));
```

```
        v_total_vanzari := f_valoare_totala_vanzari_client(v_id_client_test,  
v_data_start_test, v_data_end_test);
```

```
        dbms_output.put_line('Rezultat: Valoarea totala a vanzarilor este: ' ||  
v_total_vanzari);
```

exception

```
    when no_data_found then
```

```
        dbms_output.put_line('EROARE: Clientul cu ID ' || v_id_client_test || ' nu  
a fost gasit.');
```

end;

/

```
exception
  when no_data_found then
    dbms_output.put_line('EROARE: Clientul cu ID ' || v_id_client_test || ' nu a fo
end;
/
```

Script Output x Query Result x

Task completed in 0.264 seconds

Function F_VALOARE_TOTALA_VANZARI_CLIENT compiled

Se calculeaza valoarea vanzarilor pentru client ID: 1004 intre 25-04-2025 si 25-05-2025
Rezultat: Valoarea totala a vanzarilor este: 0

PL/SQL procedure successfully completed.

- Pachet pentru gestiunea farmaciei. Pachetul va conține subprograme pentru gestionarea medicamentelor (actualizare stoc cu o valoare dată ca parametru - procedură, afișarea informațiilor despre medicamente - procedură, modificarea prețului unui medicament cu un procent dat - procedură), stocurilor (se calculează stocul medicamentelor - funcție) și a datelor de expirare (verifică ce medicamente sunt expirate - funcție).

set serveroutput on

create or replace package pachet_gestiune_farmacie is

```
ex_cantitate_invalida exception;
pragma exception_init(ex_cantitate_invalida, -20102);
ex_pret_invalid exception;
pragma exception_init(ex_pret_invalid, -20103);
```

```

        procedure p_actualizeaza_stoc_medicament (p_id_med in
medicamente.id_med%type, p_cantitate_adaugata in number);
        function f_calculeaza_valoare_stoc_medicament (p_id_med in
medicamente.id_med%type) return number;
        procedure p_afiseaza_info_medicament (p_id_med in
medicamente.id_med%type);
        function f_verifica_expirare_medicament (p_id_med in
medicamente.id_med%type) return boolean;
        procedure p_modifica_pret_medicament (p_id_med in
medicamente.id_med%type, p_procent_modificare in number);
end pachet_gestiune_farmacie;
/

```

create or replace package body pachet_gestiune_farmacie is

```

        procedure p_actualizeaza_stoc_medicament (p_id_med in
medicamente.id_med%type, p_cantitate_adaugata in number) is
        v_id medicamente.id_med%type;
begin
        if p_cantitate_adaugata < 0 then
                raise ex_cantitate_invalida;
        end if;
        select id_med into v_id from medicamente where id_med =
p_id_med;

        update medicamente
        set stoc_med = stoc_med + p_cantitate_adaugata
        where id_med = p_id_med;
        dbms_output.put_line('stoc actualizat pentru medicamentul cu id ' ||
p_id_med || '. cantitate adaugata: ' || p_cantitate_adaugata);

exception
        when no_data_found then

```



```
dbms_output.put_line('eroare  
(p_actualizeaza_stoc_medicament): medicamentul cu id ' || p_id_med || '  
nu exista.');
```

```
when ex_cantitate_invalida then
```

```
dbms_output.put_line('eroare  
(p_actualizeaza_stoc_medicament): cantitatea adaugata nu poate fi  
negativa.');
```

```
end p_actualizeaza_stoc_medicament;
```

```
function f_calculeaza_valoare_stoc_medicament (p_id_med in  
medicamente.id_med%type) return number is
```

```
v_valoare_stoc number;
```

```
v_stoc medicamente.stoc_med%type;
```

```
v_pret medicamente.pret_med%type;
```

```
begin
```

```
select stoc_med, pret_med
```

```
into v_stoc, v_pret
```

```
from medicamente
```

```
where id_med = p_id_med;
```

```
v_valoare_stoc := v_stoc * v_pret;
```

```
return v_valoare_stoc;
```

```
exception
```

```
when no_data_found then
```

```
dbms_output.put_line('eroare  
(f_calculeaza_valoare_stoc_medicament): medicamentul cu id ' ||  
p_id_med || ' nu exista.');
```

```
end f_calculeaza_valoare_stoc_medicament;
```

```
procedure p_afiseaza_info_medicament (p_id_med in  
medicamente.id_med%type) is
```

```
rec_medicament medicamente%rowtype;
```

```
begin
```

```

select * into rec_medicament
from medicamente
where id_med = p_id_med;

        dbms_output.put_line('informatii medicament cu id: ' ||
rec_medicament.id_med);
        dbms_output.put_line(' nume: ' || rec_medicament.nume_med);
                dbms_output.put_line('          producator: ' ||
rec_medicament.producator_med);
        dbms_output.put_line(' pret: ' || rec_medicament.pret_med);
        dbms_output.put_line(' stoc: ' || rec_medicament.stoc_med);
                dbms_output.put_line('          data   expirare: ' ||
to_char(rec_medicament.data_expirare_med, 'dd-mon-yyyy'));
exception
    when no_data_found then
        dbms_output.put_line('eroare (p_afiseaza_info_medicament):
medicamentul cu id ' || p_id_med || ' nu exista.');
```

```

end p_afiseaza_info_medicament;

function    f_verifica_expirare_medicament    (p_id_med    in
medicamente.id_med%type) return boolean is
    v_data_expirare medicamente.data_expirare_med%type;
begin
    select data_expirare_med
    into v_data_expirare
    from medicamente
    where id_med = p_id_med;

    if v_data_expirare < trunc(sysdate) then
        return true;
    else
        return false;
    end if;

```

```

exception
    when no_data_found then
        dbms_output.put_line('eroare (f_verifica_expirare_medicament):
medicamentul cu id ' || p_id_med || ' nu exista.');
```

```

    end f_verifica_expirare_medicament;

    procedure p_modifica_pret_medicament (p_id_med in
medicamente.id_med%type, p_procent_modificare in number) is
```

```

        v_pret_curent medicamente.pret_med%type;
```

```

        v_pret_nou medicamente.pret_med%type;
```

```

begin
```

```

    select pret_med
```

```

    into v_pret_curent
```

```

    from medicamente
```

```

    where id_med = p_id_med;
```

```

v_pret_nou := v_pret_curent * (1 + p_procent_modificare / 100);
```

```

if v_pret_nou <= 0 then
```

```

    raise ex_pret_invalid;
```

```

end if;
```

```

update medicamente
```

```

set pret_med = round(v_pret_nou, 2)
```

```

where id_med = p_id_med;
```

```

        dbms_output.put_line('pret actualizat pentru medicamentul id ' ||
p_id_med || '. noul pret: ' || round(v_pret_nou, 2));
```

```

exception
```

```

    when no_data_found then
```

```

        dbms_output.put_line('eroare (p_modifica_pret_medicament):
medicamentul cu id ' || p_id_med || ' nu exista.');
```

```
        when ex_pret_invalid then
            dbms_output.put_line('eroare (p_modifica_pret_medicament):
modificarea procentuala face ca pretul sa fie invalid (<=0).');
        end p_modifica_pret_medicament;
```

```
end pachet_gestiune_farmacie;
/
```

```
select * from medicamente;
set serveroutput on;
```

```
declare
    v_id_med_test medicamente.id_med%type;
begin
    dbms_output.put_line('Testare pachet de gestiune farmacie');
    v_id_med_test := 28;
    begin
        pachet_gestiune_farmacie.p_afiseaza_info_medicament(p_id_med
=> v_id_med_test);
    exception
        when no_data_found then
            dbms_output.put_line(' Medicamentul cu id=' || v_id_med_test || '
nu a fost gasit.');
```

```
        end;

    begin

        pachet_gestiune_farmacie.p_actualizeaza_stoc_medicament(p_id_me
d => v_id_med_test, p_cantitate_adaugata => 3);
        dbms_output.put_line(' Stoc actualizat. Verificati manual sau
reapelati p_afiseaza_info_medicament.');
```

```

        dbms_output.put_line(' Medicamentul cu id=' || v_id_med_test || '
nu a fost gasit pentru actualizare stoc.');
```

when pachet_gestiune_farmacie.ex_cantitate_invalida then

```

        dbms_output.put_line(' Exceptie prinsa: cantitate invalida pentru
actualizare stoc.');
```

end;

end;

/

The screenshot shows the Oracle SQL Developer interface. The top toolbar contains various icons for file operations, execution, and formatting. Below the toolbar, the 'Worksheet' tab is active, displaying a PL/SQL script. The script includes a declaration for a variable `v_id_med_test` and a `begin` block. The 'Script Output' window at the bottom shows the execution results, indicating that the package and its body were compiled successfully. It also displays the output of a test procedure, which shows details for a medication with ID 28 (Vitamina D3, Zentiva, price 5, stock 1000, expiration date 01-may-2026) and confirms that the stock was updated by 3 units.

```

set serveroutput on;

declare
    v_id_med_test medicamente.id_med%type;
begin
```

Script Output x Query Result x

Task completed in 0.061 seconds

```

Package PACHET_GESTIUNE_FARMACIE compiled

Package Body PACHET_GESTIUNE_FARMACIE compiled

Testare pachet de gestiune farmacie
informatii medicament cu id: 28
  nume: Vitamina D3
  producator: Zentiva
  pret: 5
  stoc: 1000
  data expirare: 01-may-2026
stoc actualizat pentru medicamentul cu id 28. cantitate adaugata: 3
  Stoc actualizat. Verificati manual sau reapelati p_afiseaza_info_medicament.

PL/SQL procedure successfully completed.
```

- Pachet operațiuni vânzări. Pachetul va oferi funcționalități pentru înregistrarea (se înregistrează o nouă vânzare, se verifică stocul disponibil și data de expirare, actualizează stocul și calculează prețul total - procedură) și gestionarea vânzărilor de medicamente

(valoarea totală a vânzărilor efectuate către un client într-o anumită perioadă dată - funcție), inclusiv procesarea rețetelor (verifică existența și valabilitatea rețetei, procesează vânzarea medicamentelor din rețetă, dacă e compensată se aplică un procent de reducere).

create or replace package pachet_operatiuni_vanzari is

```
type rec_medicament_vanzare is record (id_medicament  
medicamente.id_med%type, cantitate number);
```

```
type tab_medicamente_vanzare is table of rec_medicament_vanzare  
index by pls_integer;
```

```
ex_medicament_indisponibil exception;
```

```
pragma exception_init(ex_medicament_indisponibil, -20203);
```

```
ex_reteta_invalida exception;
```

```
pragma exception_init(ex_reteta_invalida, -20204);
```

```
ex_vanzare_esuata exception;
```

```
pragma exception_init(ex_vanzare_esuata, -20206);
```

```
procedure p_inregistreaza_vanzare (p_id_angajat in  
angajati_farmacie.id_angajat%type,
```

```
p_id_client in clienti_farmacie.id_client%type default null,
```

```
p_lista_medicamente in tab_medicamente_vanzare,
```

```
p_id_vanzare_generat out vanzari_medicamente.id_vanzare%type,
```

```
p_total_vanzare out number);
```

```
function f_valoare_totala_vanzari_client (p_id_client in  
clienti_farmacie.id_client%type,
```

```
p_data_inceput in date,
```

```
p_data_sfarsit in date) return number;
```

```
procedure p_valideaza_si_proceseaza_reteta (p_id_reteta in  
retete.id_reteta%type,
```

```
p_id_angajat in angajati_farmacie.id_angajat%type,  
p_id_vanzare_generat out vanzari_medicamente.id_vanzare%type,  
p_total_platit_client out number);
```

```
function f_numar_medicamente_vandute_per_categorie  
(p_id_categorie in categorii_medicamente.id_categorie%type,  
p_data_inceput in date,  
p_data_sfarsit in date) return number;  
end pachet_operatiuni_vanzari;  
/
```

create or replace package body pachet_operatiuni_vanzari is

```
procedure p_inregistreaza_vanzare (p_id_angajat in  
angajati_farmacie.id_angajat%type,  
p_id_client in clienti_farmacie.id_client%type default null,  
p_lista_medicamente in tab_medicamente_vanzare,  
p_id_vanzare_generat out vanzari_medicamente.id_vanzare%type,  
p_total_vanzare out number) is  
v_id number;  
v_id_med_curent medicamente.id_med%type;  
v_cant_ceruta number;  
v_stoc_disponibil medicamente.stoc_med%type;  
v_pret_unitar medicamente.pret_med%type;  
v_medicament_expirat boolean;  
v_id_vanzare_nou vanzari_medicamente.id_vanzare%type;  
begin  
select id_angajat into v_id from angajati_farmacie where id_angajat  
= p_id_angajat;  
if p_id_client is not null then  
select id_client into v_id from clienti_farmacie where id_client =  
p_id_client;
```

```

end if;
p_total_vanzare := 0;

insert into vanzari_medicamente (data_vanzare, id_angajat,
id_client)
values (sysdate, p_id_angajat, p_id_client)
returning id_vanzare into v_id_vanzare_nou;

p_id_vanzare_generat := v_id_vanzare_nou;

if p_lista_medicamente.count > 0 then
for i in p_lista_medicamente.first .. p_lista_medicamente.last loop
v_id_med_curent := p_lista_medicamente(i).id_medicament;
v_cant_ceruta := p_lista_medicamente(i).cantitate;

if v_cant_ceruta <= 0 then
dbms_output.put_line('eroare (p_inregistreaza_vanzare):
cantitate invalida (' || v_cant_ceruta || ')
pentru medicamentul cu id ' || v_id_med_curent);
raise ex_medicament_indisponibil;
end if;

begin
select stoc_med, pret_med
into v_stoc_disponibil, v_pret_unitar
from medicamente
where id_med = v_id_med_curent;

v_medicament_expirat :=
pachet_gestiune_farmacie.f_verifica_expirare_medicament(v_id_med_
curent);

if v_medicament_expirat then

```



```

        dbms_output.put_line('eroare (p_inregistreaza_vanzare):
medicamentul id ' || v_id_med_curent || ' este expirat.');
```

raise ex_medicament_indisponibil;
 end if;

```

    if v_stoc_disponibil < v_cant_ceruta then
        dbms_output.put_line('eroare (p_inregistreaza_vanzare):
stoc insuficient pentru medicamentul cu id ' || v_id_med_curent ||
        '. cerut: ' || v_cant_ceruta || ', disponibil: ' ||
v_stoc_disponibil);
        raise ex_medicament_indisponibil;
    end if;
exception
    when no_data_found then
        dbms_output.put_line('eroare (p_inregistreaza_vanzare):
medicamentul id ' || v_id_med_curent || ' din lista nu exista sau nu a putut
fi verificat.');
```

end;

```

        p_total_vanzare := p_total_vanzare + (v_pret_unitar *
v_cant_ceruta);

        insert into rand_vanzari_med (id_vanzare, id_med, cantitate,
pret_unitar)
        values (v_id_vanzare_nou, v_id_med_curent, v_cant_ceruta,
v_pret_unitar);

        update medicamente
        set stoc_med = stoc_med - v_cant_ceruta
        where id_med = v_id_med_curent;
    end loop;
else
```

```
        dbms_output.put_line('info (p_inregistreaza_vanzare): lista de  
medicamente pentru vanzare este goala.');
```

```
    end if;
```

```
    commit;
```

```
        dbms_output.put_line('vanzare id ' || v_id_vanzare_nou || '  
inregistrata cu succes. total: ' || p_total_vanzare);
```

```
exception
```

```
    when no_data_found then
```

```
        dbms_output.put_line('eroare (p_inregistreaza_vanzare): date  
esentiale negasite (angajat, client sau medicament). vanzarea a fost  
anulata.');
```

```
        rollback;
```

```
    when ex_medicament_indisponibil then
```

```
        dbms_output.put_line('eroare (p_inregistreaza_vanzare): unul sau  
mai multe medicamente sunt indisponibile (stoc/expirat/cantitate  
invalida). vanzarea a fost anulata.');
```

```
        rollback;
```

```
    end p_inregistreaza_vanzare;
```

```
function f_valoare_totala_vanzari_client (p_id_client in  
clienti_farmacie.id_client%type,
```

```
    p_data_inceput in date,
```

```
    p_data_sfarsit in date) return number is
```

```
    v_total_valoare number := 0;
```

```
    v_id clienti_farmacie.id_client%type;
```

```
begin
```

```
    select id_client into v_id from clienti_farmacie where id_client =  
p_id_client;
```

```
    if p_data_inceput > p_data_sfarsit then
```

```

        dbms_output.put_line('eroare (f_valoare_totala_vanzari_client):
data de inceput este dupa data de sfarsit.');
```

return 0;

end if;

```

select nvl(sum(rvm.cantitate * rvm.pret_unitar), 0)
into v_total_valoare
from vanzari_medicamente vm
join rand_vanzari_med rvm on vm.id_vanzare = rvm.id_vanzare
where vm.id_client = p_id_client
and vm.data_vanzare between p_data_inceput and p_data_sfarsit;
```

```

return v_total_valoare;
exception
when no_data_found then
    dbms_output.put_line('eroare (f_valoare_totala_vanzari_client):
clientul cu id ' || p_id_client || ' nu exista.');
```

end f_valoare_totala_vanzari_client;

```

procedure p_valideaza_si_proceseaza_reteta (p_id_reteta in
retete.id_reteta%type,
p_id_angajat in angajati_farmacie.id_angajat%type,
p_id_vanzare_generat out vanzari_medicamente.id_vanzare%type,
p_total_platit_client out number) is
rec_reteta retete%rowtype;
v_lista_med_reteta tab_medicamente_vanzare;
v_idx pls_integer := 0;
v_procent_compensare number := 0;
v_id angajati_farmacie.id_angajat%type;
begin
    select id_angajat into v_id from angajati_farmacie where id_angajat
= p_id_angajat;
    select * into rec_reteta from retete where id_reteta = p_id_reteta;
```

```

if rec_reteta.data_expirare < trunc(sysdate) then
                                dbms_output.put_line('eroare
(p_valideaza_si_proceseaza_reteta): reteta id ' || p_id_reteta || ' este
expirata.');
```

raise ex_reteta_invalida;

```

end if;
```

```

for rec_med_reteta in (select id_med, cantitate from
medicamente_reteta where id_reteta = p_id_reteta) loop
    v_idx := v_idx + 1;
                                v_lista_med_reteta(v_idx).id_medicament :=
rec_med_reteta.id_med;
                                v_lista_med_reteta(v_idx).cantitate :=
rec_med_reteta.cantitate;
end loop;
```

```

if v_lista_med_reteta.count = 0 then
                                dbms_output.put_line('eroare
(p_valideaza_si_proceseaza_reteta): reteta id ' || p_id_reteta || ' nu are
medicamente asociate.');
```

raise ex_reteta_invalida;

```

end if;
```

```

p_inregistreaza_vanzare(p_id_angajat => p_id_angajat,
    p_id_client => rec_reteta.id_client,
    p_lista_medicamente => v_lista_med_reteta,
    p_id_vanzare_generat=> p_id_vanzare_generat,
    p_total_vanzare => p_total_platit_client );
```

```

if upper(rec_reteta.compensate) = 'DA' then
    v_procent_compensare := 50;
```

```

        p_total_platit_client := round(p_total_platit_client * (1 -
v_procent_compensare / 100), 2);
        dbms_output.put_line('reteta compensata. total final dupa
compensare (' || v_procent_compensare || %): ' || p_total_platit_client);
    end if;

```

```

        dbms_output.put_line('reteta id ' || p_id_reteta || ' procesata. vanzare
id: ' || p_id_vanzare_generat || '. total platit: ' || p_total_platit_client);

```

```

exception
    when no_data_found then
        dbms_output.put_line('eroare
(p_valideaza_si_proceseaza_reteta): angajat sau reteta ('||p_id_reteta||')
negasit(a).');
        rollback;
    when ex_reteta_invalida then
        dbms_output.put_line('eroare
(p_valideaza_si_proceseaza_reteta): reteta id ' || p_id_reteta || ' este
invalida (expirata sau fara medicamente).');
        rollback;
    when ex_vanzare_esuata then
        dbms_output.put_line('eroare
(p_valideaza_si_proceseaza_reteta): vanzarea asociata retetei id ' ||
p_id_reteta || ' a esuat (cauza a fost logata in p_inregistreaza_vanzare).');

end p_valideaza_si_proceseaza_reteta;

```

```

function f_numar_medicamente_vandute_per_categorie
(p_id_categorie in categorii_medicamente.id_categorie%type,
p_data_inceput in date,
p_data_sfarsit in date) return number is
v_total_unitati number := 0;
v_id categorii_medicamente.id_categorie%type;

```

```

begin
    select id_categorie into v_id from categorii_medicamente where
id_categorie = p_id_categorie;

    if p_data_inceput > p_data_sfarsit then
        dbms_output.put_line('eroare
(f_numar_medicamente_vandute_per_categorie): data de inceput este
dupa data de sfarsit.');
```

return 0;

end if;

```

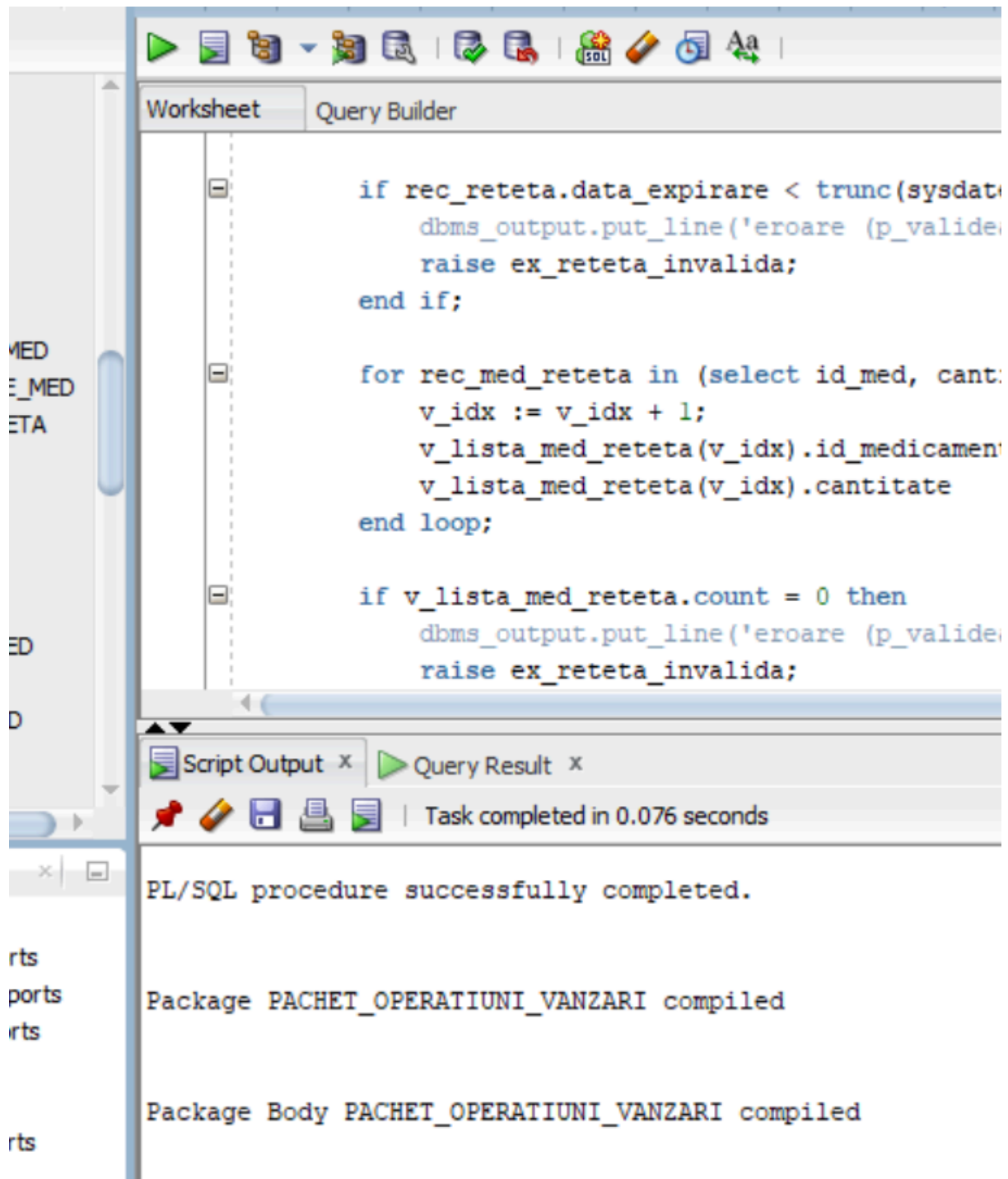
    select nvl(sum(rvm.cantitate),0)
into v_total_unitati
from rand_vanzari_med rvm
join vanzari_medicamente vm on rvm.id_vanzare = vm.id_vanzare
join medicamente m on rvm.id_med = m.id_med
where m.id_categorie = p_id_categorie
    and vm.data_vanzare between p_data_inceput and p_data_sfarsit;

    return v_total_unitati;
exception
    when no_data_found then
        dbms_output.put_line('eroare
(f_numar_medicamente_vandute_per_categorie): categoria cu id ' ||
p_id_categorie || ' nu exista.');
```

end f_numar_medicamente_vandute_per_categorie;

end pachet_operatiuni_vanzari;

/



e. Triggers

- Trigger pentru afișarea modificărilor de preț la medicamente.

create or replace trigger trg_afiseaza_modificare_pret

after update of pret_med on medicamente

for each row

when (new.pret_med != old.pret_med)

declare

 v_utilizator varchar2(100);

begin

 begin

 select user into v_utilizator from dual;

 exception

 when others then

 v_utilizator := 'necunoscut';

 end;

 -- Afisam informatia despre modificare

dbms_output.put_line('-----
----');

dbms_output.put_line('Modificare pret pentru medicament.');

dbms_output.put_line(' ID Medicament: ' || :old.id_med);

dbms_output.put_line(' Nume Medicament: ' || :old.nume_med);

dbms_output.put_line(' Pret Vechi: ' || :old.pret_med);

dbms_output.put_line(' Pret Nou: ' || :new.pret_med);

dbms_output.put_line(' Modificat de: ' || v_utilizator);

 dbms_output.put_line(' Data Modificare: ' || to_char(sysdate,
'dd-mon-yyyy hh24:mi:ss'));

dbms_output.put_line('-----
----');

end trg_afiseaza_modificare_pret;

/

update medicamente

set pret_med=pret_med-1;


```
end trg_afiseaza_modificare_pret;  
/  
update medicamente  
set pret_med=pret_med-1;
```

Script Output x Query Result x

Task completed in 0.139 seconds

Trigger TRG_AFISEAZA_MODIFICARE_PRET compiled

Modificare pret pentru medicament.
ID Medicament: 1
Nume Medicament: Amoxicilina
Pret Vechi: 10
Pret Nou: 9
Modificat de: PADUREANUM_67
Data Modificare: 25-may-2025 18:28:45

Modificare pret pentru medicament.
ID Medicament: 2
Nume Medicament: Paracetamol
Pret Vechi: 5

➤ Trigger pentru setarea automată a datei de angajare.

```
create or replace trigger trg_set_data_angajare  
before insert on angajati_farmacie  
for each row  
begin
```

```

if :new.data_angajare is null then
    :new.data_angajare := trunc(sysdate);
end if;
end trg_set_data_angajare;
/

```

➤ Trigger pentru a preveni ștergerea furnizorilor cu achiziții existente.

```

create or replace trigger trg_prevenire_stergere_furnizor
before delete on furnizori_med
for each row
declare
    v_nr_achizitii number;
begin
    select count(*)
    into v_nr_achizitii
    from achizitii_medicamente
    where id_furnizor = :old.id_furnizor;

    if v_nr_achizitii > 0 then
        raise_application_error(-20001, 'furnizorul ' || :old.nume_furnizor ||
            ' (id: ' || :old.id_furnizor || ') nu poate fi sters deoarece
are ' ||
            v_nr_achizitii || ' achizitii inregistrate.');
```

```

    end if;
end trg_prevenire_stergere_furnizor;
/

delete from furnizori_med
where upper(nume_furnizor) = 'PFIZER';

```

```
delete from furnizori_med  
where upper(nume_furnizor) = 'PFIZER';
```

Script Output x

Query Result x

Task completed in 0.414 seconds

Trigger TRG_PREVENIRE_STERGERE_FURNIZOR compiled

Error starting at line : 20 in command -

delete from furnizori_med

where upper(nume_furnizor) = 'PFIZER'

Error at Command Line : 20 Column : 13

Error report -

SQL Error: ORA-20001: furnizorul Pfizer (id: 124) nu poate fi sters deoarece are 1 achizitie inregistrata.

ORA-06512: at "PADUREANUM_67.TRG_PREVENIRE_STERGERE_FURNIZOR", line 10

ORA-04088: error during execution of trigger 'PADUREANUM_67.TRG_PREVENIRE_STERGERE_FURNIZOR'